UNIVERSIDAD
NEBRIJA

MASTER IN QUANTUM COMPUTING

Quantum Error Correction Decoders for 2D Color Codes:

Systematic Simulation and Comparison

Antonio Morales García

Scientific Director: Dr. Miguel Ángel Martín-Delgado Alcántara
Tutor: Prof. Francisco Gálvez Ramírez

Academic Year 2025/2026

Pendiente: dedicatoria a las personas
a las que más quiero, las que más me quieren.

Acknowledgments

[Pendiente] Agradecimientos a todos los que han hecho posible que haga este TFM.

Nota para no olvidar - Mis padres, que me inculcaron la curiosidad, la capacidad de cuestionar lo establecido y la perseverancia - Mis abuelos, que me inculcaron el amor propio (hacer las cosas bien) y me enseñaron a valorar el conocimiento y la sabiduría - Mi mujer y mis hijos, que han sacrificado casi tanto como yo para poder hacer esto (hacer un TFM pasados los 40 requiere grandes sacrificios familaires) - Mis mentores y profesores, que han ido construyendo el conocimiento que me ha permitido llegar a donde he llegado, y que me han inspirado para adentrarme en el mundo de la computación cuántica – los profesores son arquitectos invisibles de nuestra identidad - Y, el último y más importante, Miguel Ángel, por este privilegio

Structure of this thesis

Indexes and Tables of Content

A brief Table of Contents is provided at the beginning of this work (see page viii), outlining only the chapters and main sections.

A comprehensive Table of Contents can be found at the end of the thesis (page I), alongside the List of Figures (page III) and the Bibliography (page VII).

Source code and repository structure

Chapters 5 and 6 show the result of multiple simulations and code execution.

Small code snippets are shown to illustrate how the different tools and libraries work. The complete code required to execute the simulations presented in this work is available in a git repository at `https://github.com/othermore/TFM_QEC_2D_color_code_decoders`. **The line numbers in the original files for each snippet are displayed** to facilitate locating them in the source files.

The code is located in the `src` directory. Each chapter has its own directory and python notebook. The specific code for each simulation is located in separate files (starting with `s` and the number of the simulation) and included in the notebook.

Libraries and resources required for multiple simulations within a given chapter, are included in the chapter's directory.

Overview

CHAPTER 1

Introduction: Classical error correction and foundations of quantum error correction

The field of quantum computing is evolving rapidly. It has transitioned from a theoretical curiosity to being generally accepted as a frontier technology. The fundamental premise of this discipline is that information processing based on the laws of quantum mechanics can solve problems that remain computationally out of reach for classical computing [1]. This paradigm began in 1982, when Richard Feynman pointed out that classical computers are not well equipped to simulate quantum systems due to the exponential growth of space required to represent a quantum state. He proposed that a computer based on quantum mechanical principles would be required to model nature effectively [2].

The Quantum Advantage and the Noise Barrier

The true potential of this technology was demonstrated in 1994 by Peter Shor, who developed a quantum algorithm for integer factorization in polynomial time [3]. Shor's work proved that quantum computing could break widely used cryptographic schemes, such as RSA. That is a theoretical proof of what has been called "Quantum Advantage". However, this assumes the existence of quantum hardware that has not been realized yet.

The realization of such a computer is difficult because of the fragility of quantum states. Qubits are vulnerable to noise from environmental interactions and to inaccuracies in the operations we make with them. When this noise affects the qubits, the information they store is lost.

We refer to this reality as the era of Noisy Intermediate-Scale Quantum (NISQ) technology. In this era, error rates in quantum gates are several orders of magnitude higher than their classical counterparts. Without a robust way to protect quantum information, even the most advanced algorithms fail before reaching a meaningful result.

The Evolution of Quantum Error Correction

As we will see later in this chapter, unlike classical bits, quantum information cannot be protected by just copying it, due to the no-cloning theorem. To overcome this, the field of Quantum Error Correction (QEC) was pioneered by Peter Shor [4] and Andrew Steane [5]. They demonstrated that a single "logical" qubit can be represented with multiple "physical" qubits. More formally, the logical qubit is encoded in a larger Hilbert space of multiple physical qubits, allowing for the detection and correction of errors without destroying the underlying quantum state.

While Shor's solution (known as Shor's code) proves that QEC can be theoretically implemented, it is unable to effectively mitigate error, as the added complexity of the code causes more errors than it can prevent.

A significant milestone in QEC was the introduction of topological codes by Alexei Kitaev [6], which will be discussed in Chapter 2. Kitaev's Toric Code and subsequent Surface Codes presented a new paradigm of local, lattice-based architectures where errors are detected by measuring local parities (stabilizers). These types of codes became leading candidates for scalable hardware due to their high error thresholds and two-dimensional physical layouts, which map well to the most common realizations of quantum computers.

1.1 Project Goals and Scope

This Master's Thesis focuses on the simulation and comparative analysis of decoders for the 2D triangular color code. Introduced by Bombín and Martin-Delgado [7], color codes represent an evolution over surface codes that offers significant advantages, particularly regarding the transversal implementation of logical gates, which simplifies fault-tolerant operations.

The performance of various QEC protocols and their corresponding decoders has been extensively studied by multiple authors. However, the community currently lacks accessible, consolidated resources that establish a clear methodology and a unified framework for evaluating these results. To address this gap, this work aims not merely to provide a performance analysis, but to serve as a detailed methodological guide. By offering a structured evaluation framework, it seeks to ease the steep learning curve for new researchers and graduates starting their work in quantum error correction.

Scope

The scope of this work is defined by three specific constraints regarding the lattice geometry, how to evaluate decoding algorithms, and the noise model. The foundations of these concepts are detailed in the following sections and chapters, starting with an overview of classical error correction principles. Below, we outline the specific constraints.

As this introduction establishes the boundaries of the research, it references advanced concepts (like topological geometries or noise models) that will be formally defined in subsequent chapters.

First, as we will formally define in Chapter 2, color codes can be implemented through several geometries (4.8.8, 6.6.6 and 4.6.12). In this work, we will specifically adopt a **4.8.8 lattice geometry**. This choice is motivated by its lower qubit requirements (fewer qubits are needed per distance) and its properties as a doubly-even code, which enables transversal implementation of the full Clifford group, including H, $S^\dagger$ and CNOT [7].

Second, our objective is to quantify the performance of various decoding strategies, from classical **Projection decoders** to state-of-the-art **Möbius** and **Concatenated MWPM** approaches. Using advanced simulation and sampling tools such as `stim` [8] and `sinter`, in Chapter 7 we will compare their efficiency through standard metrics:

physical error thresholds (p_{th}), logical failure rate scaling (p_L) as a function of physical noise, theoretical sub-threshold suppression limits, and the trade-off between decoding accuracy and computational time.

Finally, as we will discuss in Chapter 3, various sources of noise affect real quantum hardware. While realistic hardware-aware simulations of QEC codes require accounting for all of them—including imperfect gates and measurement errors—this study focuses strictly on **code-capacity noise**. This foundational model isolates the decoherence of data qubits over time by assuming perfect syndrome extraction. Adopting this approach allows us to evaluate the strengths of the 4.8.8 lattice and the algorithmic efficiency of the decoders, without the confounding variables introduced by circuit-level operations.

1.2 Classical error correction

Before discussing how to protect quantum information, it is essential to understand the basic mechanisms of classical error correction. While classical bits are physically robust, they are still susceptible to occasional errors (e.g. caused by hardware degradation or electromagnetic interference). Since classical systems are naively discrete (information is binary), a classical error is strictly discrete: a bit flips from 0 to 1, or from 1 to 0.

To detect and correct these bit-flips, classical computing relies on the concept of **redundancy**. By adding extra bits to the original message, we can deduce if an error has occurred and, in many cases, revert it. Two of the most foundational techniques used to achieve this are repetition codes and parity checks. These classical concepts serve as the baseline for building analogous quantum error correction solutions.

Repetition Codes and Parity Checks

The simplest approach to redundancy is the **repetition code**. If we want to send a single logical bit, we copy it multiple times. For example, in a 3-bit repetition code, a logical 0 is encoded as 000, and a logical 1 as 111.

If a noise event flips one of the physical bits (e.g., the state becomes 010), the receiver can apply a **majority vote** to determine the original message. Since there are more zeros than ones, the system concludes the intended logical state was 0, effectively correcting the error.

For the 3-bit repetition code to be useful, the probability of a single bit flipping, p, must be low enough that the chance of two or three bits flipping simultaneously ($3p^2(1-p)+p^3$) is negligible.

Figure 1.1. *Mechanism of a classical 3-bit repetition code correcting a single bit-flip error.*

A more efficient method is the use of **parity checks**. Instead of copying the entire message, we calculate a parity bit based on the sum of the data bits. For instance, we can add a bit to ensure that the total number of 1s in a string is always an even number. If a single bit flips during transmission, the overall parity becomes odd, immediately alerting the system to the presence of an error. By structuring multiple parity checks across different overlapping blocks of data, classical systems can locate and correct errors with minimal overhead.

This overlapping block parity concept is the foundation of classical Hamming codes, which can mathematically pinpoint exactly which bit flipped without needing massive repetition.

The Three Major Challenges of Quantum Error Correction

It is natural to assume we could simply apply these classical techniques to quantum computers. However, translating repetition and parity concepts to the quantum realm is

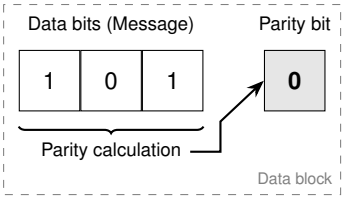


Figure 1.2. *A parity check bit appended to a 3-bit message. The parity bit (0) is calculated to ensure that the total number of '1's in the data block is even.*

not straightforward. Quantum mechanics imposes three fundamental constraints that classical computing does not face:

1. **The No-Cloning Theorem:** Classical repetition codes require copying the information. In quantum mechanics, it is fundamentally impossible to create an identical copy of an unknown arbitrary quantum state [9]. Therefore, we cannot simply duplicate a qubit to create redundancy.
2. **Measurement Destroys Information:** Classical parity checks require reading the bits to see if an error occurred. If we attempt to measure a qubit in a superposition state to check for errors, the superposition collapses into a definitive classical state. Measuring the data to find the error destroys the quantum information we are trying to protect.
3. **Continuous Errors:** Classical computers are binary (a bit is either 0 or 1) and discrete (there are no intermediary values). This property applies to classical errors (a bit is either right or wrong). Qubits, however, exist in a continuous state space, the complex Hilbert space. This means that there is an infinite number of possible errors that can occur, making it apparently impossible to design a code that accounts for every potential continuous deviation.

A qubit state, $|\psi\rangle$, is a vector in a two-dimensional complex Hilbert space, defined by continuous amplitudes $\alpha|0\rangle + \beta|1\rangle$, with α and β being complex numbers. Such state can be represented in the Bloch Sphere. Because of this continuity, an error can be an arbitrarily small rotation.

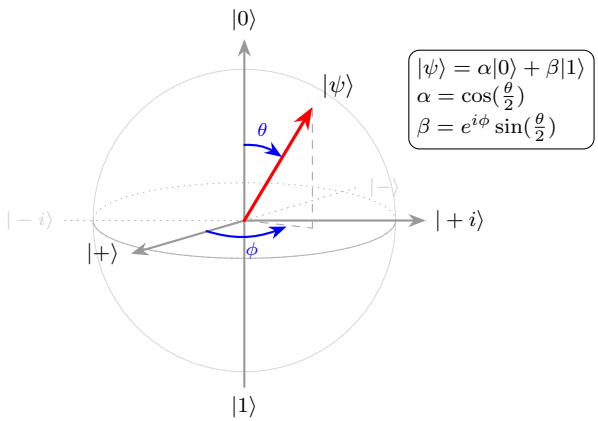


Figure 1.3. *Geometric representation of a qubit on the Bloch sphere. The parameters θ and ϕ define the continuous orientation of the state vector $|\psi\rangle$ relative to the computational basis states $|0\rangle$ and $|1\rangle$.*

The solution to these three fundamental barriers was formulated by Peter Shor, leading

to the creation of the first true quantum error correction code, which we explore in the next section.

1.3 Shor code

The transition from classical to quantum error correction was made possible by Peter Shor's 1995 proposal [4]. His approach demonstrated that the three barriers of quantum mechanics mentioned above could be bypassed using entanglement, parity-based measurements and concatenation of QEC codes.

The 3-qubit Code: Solving Cloning and Measurement

The first step is understanding that while we cannot copy an unknown state $|\psi\rangle$, we can **distribute** its information across an entangled system. In the 3-qubit bit-flip code, the encoding does not create copies; it creates a multi-particle state where the logical information is stored in a correlation of these particles' states.

The 3-qubit codes exposed here, which are simple repetition codes, are the main components of the 9-qubit Shor code. It is important to note that in Shor's original paper [4] these 3-qubit codes were not introduced as independent, standalone protocols.

$$|0\rangle_L = |000\rangle, \quad |1\rangle_L = |111\rangle$$

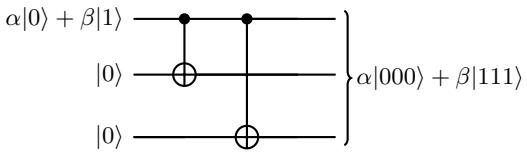


Figure 1.4. *Encoding circuit for the bit-flip code. Entanglement distributes the logical information without violating the no-cloning theorem.*

To solve the measurement problem, Shor introduced the concept of **syndrome extraction**. Instead of measuring each qubit (which would destroy the superposition $\alpha|000\rangle + \beta|111\rangle$), we compare pairs of these qubits to determine whether they are equal or different without gaining any information about their actual state.

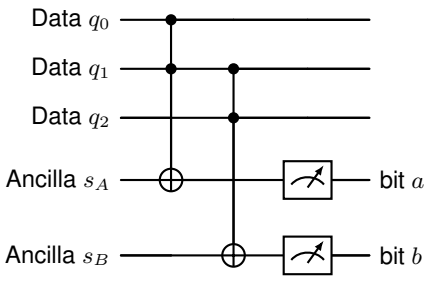


Figure 1.5. *Syndrome measurement: The ancilla qubits are activated when the two qubits that they monitor are different. Depending on which of the two ancilla qubits are activated, we can determine which of the qubits has flipped. This is known as a Z-type detector*

As shown in Figure 1.5, measuring the ancilla qubits does not provide information about the state of each individual qubit, but it provides information about two qubits being different. The following table shows how the measuring of these ancilla qubits maps to specific identified errors.

Syndrome (s_A)	Syndrome (s_B)	Error Location	Correction
0	0	None	No action
1	0	q_0	Apply X_0
0	1	q_2	Apply X_2
1	1	q_1	Apply X_1

A Code to Detect Phase-flip Errors

The bit-flip code addresses X errors. The other fundamental quantum error is the phase-flip (Z), which changes the relative phase (the sign) in a superposition: $Z(\alpha|0\rangle + \beta|1\rangle) = \alpha|0\rangle - \beta|1\rangle$. This is strictly a quantum phenomenon and has no classical counterpart.

Formally, the Hadamard gate maps the standard computational basis $\{|0\rangle, |1\rangle\}$ to the superposition basis $\{|+\rangle, |-\rangle\}$. In terms of operators, $HZH = X$.

To correct a phase-flip, we can leverage a key property of quantum gates: a Hadamard gate (H) transforms a Z (phase-flip) error into an X error (bit-flip). Using the H gate, we can reuse the logic of the bit-flip code, where an X-parity check (i.e. comparing pairs of qubits) in the logical space effectively functions as a Z-check in the physical space.

The logical states are thus defined as:

$$|0\rangle_L = |+++\rangle, \quad |1\rangle_L = |---\rangle$$

The encoding circuit entangles the qubits using the CNOT structure of the bit-flip code and then applies a global Hadamard transformation to switch the entire block into the relevant basis.

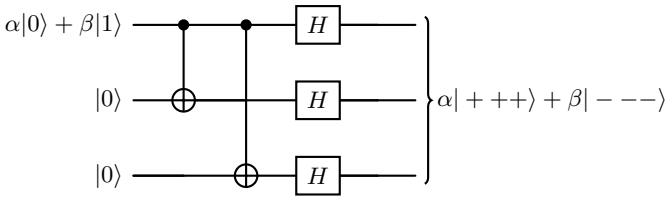


Figure 1.6. *Encoding circuit for the 3-qubit phase-flip code. The information is first distributed via entanglement, and then Hadamard gates transform the final state into the $\{|+\rangle, |-\rangle\}$ basis.*

To measure the relative phases of the data qubits (implementing a X-type detector), the phase-kickback technique is used: the ancilla is put on a $|+\rangle$ state and CNOT gates are run with the ancilla as control qubit.

Applying the same measuring technique to these three qubits in pairs, allows determining the syndrome, which in this case will point to a Z error.

The 9-qubit Shor Code: Correcting Arbitrary Errors

The 3-qubit code provides a solution to the first two challenges, but it only corrects one type of error (either bit-flip or phase-flip). To be able to correct flips in any of the three axis, Shor's proposed the **concatenation** of a bit-flip code and a phase-flip code. By nesting them, we create a 9-qubit block that protects against both bit-flips (X) and phase-flips (Z) simultaneously.

The logical basis states are constructed by taking three clusters of qubits. Each cluster is in a superposition ($|000\rangle \pm |111\rangle$) to handle phase errors, while each individual qubit within the cluster is tripled to handle bit-flip errors:

$$|0\rangle_L = \frac{(|000\rangle + |111\rangle)(|000\rangle + |111\rangle)(|000\rangle + |111\rangle)}{\sqrt{8}}$$

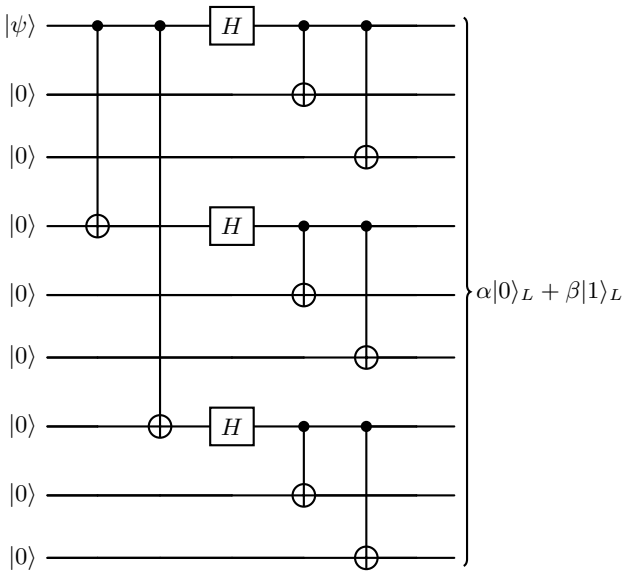


Figure 1.7. *Full encoding circuit for the 9-qubit Shor code. The initial CNOT and Hadamard gates encode the logical state into a 3-qubit phase-flip code. Subsequently, each of these three qubits is encoded into a 3-qubit bit-flip code, resulting in the final 9-qubit state.*

$$|1\rangle_L = \frac{(|000\rangle - |111\rangle)(|000\rangle - |111\rangle)(|000\rangle - |111\rangle)}{\sqrt{8}}$$

To extract the syndrome of the 9-qubit Shor code, a total of 8 ancilla qubits are needed, 6 of them monitoring pairs of qubits to detect bit flips inside the clusters (Z-type detectors), and 2 of them monitoring full clusters to detect phase flips between two clusters (X-type detectors). The Table 1.1 shows the ancilla qubits and the data qubits they monitor.

Ancilla	Detector type	Targeted qubits	Error detected
a_1	Z-type	q_1, q_2	Bit-flip (X) in Block 1
a_2	Z-type	q_2, q_3	Bit-flip (X) in Block 1
a_3	Z-type	q_4, q_5	Bit-flip (X) in Block 2
a_4	Z-type	q_5, q_6	Bit-flip (X) in Block 2
a_5	Z-type	q_7, q_8	Bit-flip (X) in Block 3
a_6	Z-type	q_8, q_9	Bit-flip (X) in Block 3
a_7	X-type	$q_1, q_2, q_3, q_4, q_5, q_6$	Phase-flip (Z) in clusters 1 or 2
a_8	X-type	$q_4, q_5, q_6, q_7, q_8, q_9$	Phase-flip (Z) in clusters 2 or 3

Table 1.1. *Ancilla qubits to extract the syndrome in the 9-qubit Shor code.*

The Table 1.2 shows some examples of possible measured syndromes, the corresponding errors and the necessary recovery operation.

It is important to note that phase-flip correction can be applied to any qubit of the cluster for which we detected the error. This is because the phase is encoded in the relative phase between the $|000\rangle$ and $|111\rangle$ components of the cluster, which changes by changing the phase of any of the three qubits. To correct the Y error, the combination of X and Z can be used.

Syndrome (Z-type) (a_1a_2, a_3a_4, a_5a_6)	**Syndrome (X-type)** (a_7, a_8)	**Error Type**	**Correction**
$(00, 00, 00)$	$(0, 0)$	None	I (No error)
$(10, 00, 00)$	$(0, 0)$	X_1	Apply X_1
$(11, 00, 00)$	$(0, 0)$	X_2	Apply X_2
$(01, 00, 00)$	$(0, 0)$	X_3	Apply X_3
$(00, 00, 00)$	$(1, 0)$	Z in cluster 1	Apply Z_1
$(00, 00, 00)$	$(1, 1)$	Z in cluster 2	Apply Z_4
$(00, 00, 00)$	$(0, 1)$	Z in cluster 3	Apply Z_7
$(10, 00, 00)$	$(1, 0)$	Y_1	Apply Y_1

Table 1.2. *Decoding the syndrome in the 9-qubit Shor code.*

The Discretization of Quantum Errors

We have seen that the 9-qubit Shor code enables correcting both X and Z, but how can we correct an infinite number of possible small rotations?

The solution is surprisingly simple. It relies on the mathematics of quantum measurement. As proven by Knill and Laflamme [10], we do not actually need to correct any arbitrary error. Any arbitrary single-qubit error can be expressed as a linear combination of the Pauli matrices: I (identity, no error), X (bit-flip), Z (phase-flip) and Y (both bit and phase-flip).

By designing a quantum error correction code capable of detecting and correcting independent X and Z errors, such as the one designed by Shor, the act of measuring the error (the syndrome extraction) forces the continuous, arbitrary error to collapse into one of the discrete Pauli operators (I, X, Y and Z). Since I means no error and the Pauli Y operator is simply a combination of X and Z ($Y = iXZ$), **correcting X and Z is sufficient to correct any arbitrary error**. This phenomenon, known as the discretization of quantum errors, effectively reduces the continuity quantum error problem back to a discrete problem analogous to classical bit-flips.

Error Distinguishability

The fundamental requirement for a quantum error correction code is that errors that require different recovery operations must result in distinct syndromes. In formal terms, this means that the errors must map the logical state into orthogonal, distinguishable subspaces. If two physical errors produce the same syndrome but result in fundamentally different corrupted states, the decoder will not know which correction to apply, leading to a logical failure.

*Any qubit within a cluster suffering a phase-flip error in the 9-qubit Shor code results in the same syndrome. However, an operation on any of the three qubits of the cluster will fix the state of the logical qubit. This is known as degeneracy, and we say that the three phase-flip errors that can affect a cluster are **degenerated errors**.*

The 9-qubit Shor code works well precisely because it separates bit-flip and phase-flip errors into distinct, non-overlapping parity checks. For example, any bit-flip error in a single qubit will activate either one or two of the first 6 ancillas (see Table 1.2), which can map uniquely to the specific qubit that flipped.

Analyzing these subspaces manually becomes unsustainable for larger, more complex quantum codes. This scalability problem motivates the need for a robust, algebraic framework.

1.4 The Stabilizer Formalism

As quantum codes scale, representing the logical state as a vector of 2^n amplitudes becomes computationally intractable. Tracking the evolution of a highly entangled state qubit by qubit is unsustainable. The Stabilizer Formalism, introduced by Daniel Gottesman in 1997 [11], offers an elegant workaround: instead of describing the quantum state directly, we describe the set of operators that leave the state unchanged. More specifically, we focus on the operators that leave unchanged a valid state (i.e. a state where no error has occurred) but modify an erroneous state.

Stabilizer Operators

The core concept of this framework relies on a simple mathematical condition. We say that an operator S **stabilizes** a quantum state $|\psi\rangle$ if applying S to the state does not alter it in any way:

$$S|\psi\rangle = |\psi\rangle \tag{1.1}$$

In linear algebra, an eigenvector $|\psi\rangle$ of an operator S satisfies $S|\psi\rangle = \lambda|\psi\rangle$, where λ is a constant called the eigenvalue. In the stabilizer formalism, we strictly look for states where $\lambda = +1$. Because the state remains unchanged, measuring S will yield a deterministic outcome of $+1$, indicating no error. It's important to note that measuring the $+1$ usually results in a 0 value for the classical syndrome bit.

In practice, a stabilizer operator is simply a combination of Pauli matrices (X, Y, Z) acting on a specific subset of qubits. For example, an operator Z_1Z_2 monitors the phase relationship of the first and second qubits. These operators are what we referred to as "syndrome detectors" or "parity checks" in the previous section when discussing the Shor code.

Note that an error affecting the qubits can also be represented as an operator (e.g. a bit-flip is a X Pauli matrix applied to the flipping qubit).

We can **measure** a stabilizer operator. Such measurement is implemented as a parity check on the data qubits that are connected to this operator, with the result being reflected in the state of an ancilla qubit:

- Z-type stabilizers, which detect bit-flip errors, are measured with a CNOT gate that has the data qubits as source and the ancilla qubit as target.
- X-type stabilizers, which detect phase-flip errors, are measured with a CNOT gate that has the data qubits as target and the ancilla qubit as source, which produces a *phase kickback*.

We must also introduce the concept of **commutation**. In the mathematics of Pauli matrices, two operators either commute (meaning their order of application does not matter) or **anti-commute** (meaning swapping their order introduces a minus sign). Because both stabilizers and errors are Pauli operators, we can determine commutation properties for them.

Mathematically, two operators A and B commute if $AB = BA$, and anti-commute if $AB = -BA$. For Pauli matrices, identical types commute (e.g., an X error commutes with an X stabilizer), but different types acting on the same qubit anti-commute (e.g., an X error anti-commutes with a Z stabilizer).

The algebraic property of commutation is the core mechanism of error detection. If a quantum system is stabilized by an operator S, we can periodically measure S to check the health of the qubits connected to it:

- If no error has occurred, or if an error occurs that *commutes* with S, the state remains a valid $+1$ eigenstate. The measurement returns $+1$, telling the decoder that those specific qubits are safe (or not affected by an error detectable by S).
- If a physical error occurs that *anti-commutes* with S, the error flips the sign of the state. The measurement will return -1, acting as a localized alarm that an error has corrupted the monitored qubits.

We can see that if we can construct a set of operators that can detect all the possible errors affecting our physical qubits, we can determine the operations required to correct them.

The Stabilizer Group and Generators

*A **group** is a set equipped with an operation that combines any two elements to form a third element within the same set. The Pauli group is closed under matrix multiplication (the product of two Pauli operators is always a Pauli operator).*
*A group whose elements commute between them is known as an **abelian group**.*

To build these stabilizer operators, we use the n-qubit Pauli group, denoted as $\mathcal{P}_n$. This group consists of all possible tensor products of the standard Pauli matrices (I, X, Y, Z) applied to n qubits, multiplied by a global phase factor $(\pm 1, \pm i)$.

Under this formalism, a quantum error correction code is defined by its **Stabilizer Group** $(\mathcal{S})$, which is a commuting subgroup of $\mathcal{P}_n$, that is, a subset of $\mathcal{P}_n$ for which all its members commute between them.

The reason why this is a strict requirement is that, for a QEC code to be valid, all the operators in the group must share a common set of eigenvectors with eigenvalue $+1$. These eigenvectors are the logical states of the QEC code, and are not affected by any of the stabilizer operators. Only commuting operators can share a common set of eigenvectors.

Additionally, for $\mathcal{S}$ to define a valid code, the operator $-I$ must not be an element of the group. If $-I$ was an element of $\mathcal{S}$, then we would have $-I|\psi\rangle = |\psi\rangle$. This only holds if $|\psi\rangle = 0$, meaning the code would have no valid quantum states.

We've defined the $\mathcal{S}$ as a set of commuting operators. However, we do not need every single operator in $\mathcal{S}$ to define the code. We only need a small, independent set of operators called **generators**. Just as a three basis vectors can define an entire 3D space, multiplying these generators together produces all the other stabilizers in the group. This set of independent operators represents the set of detectors that we will have to implement in the quantum circuit.

For an n-qubit system, choosing $n - k$ independent commuting generators uniquely defines a QEC code capable of encoding k qubits. Such generators create a protected logical subspace with 2^k logical states.

Standard [[n, k, d]] Notation

Note that the parameter n does not include the number of ancilla qubits required to extract the syndrome, as it depends on the actual implementation.

In the field of Quantum Error Correction, stabilizer codes are conventionally described using the $[[n, k, d]]$ notation, which provides a summary of the code's cost and capacity:

- n **(Physical Qubits):** The total number of physical qubits used to construct the code.
- k **(Logical Qubits):** The number of logical qubits encoded.
- d **(Code Distance):** The minimum number of physical errors required to cause an undetected logical failure.

Under this convention, the Shor code is formally classified as a $[[9, 1, 3]]$ code, meaning it uses nine physical qubits to protect one logical qubit with a distance of three, and requires eight $(n - k)$ stabilizers.

The distance directly determines the error-handling capabilities of the code through two fundamental rules:

1. **Detection:** A code can detect any error affecting up to $d - 1$ physical qubits.
2. **Correction:** A code can correct up to t physical errors:

$$t = \left\lfloor \frac{d-1}{2} \right\rfloor \tag{1.2}$$

For the Shor code $(d = 3)$, these rules imply that the code can detect up to 2 errors and correct $t = 1$ error. If two errors occur simultaneously, the syndrome would cause a wrong correction. This is a permanent trade-off in QEC: increasing the correction threshold t, requires increasing the distance d, which requires more physical qubits n.

1.5 The Threshold Theorem

For a QEC code to effectively mitigate the errors, the error introduced by the overhead QEC in the circuit must be lower than the error effectively corrected, so the resulting logical error is below the physical error rate.

The distance of a code relates to the capacity of the code to reduce the error rate, but it is also correlated with the code overhead: to increase the distance, we need to add more physical qubits, n, to codify the same number of logical qubits, k.

As we increase the complexity of quantum computations, we need quantum circuits that are wider (more qubits) and deeper (more steps in the calculation). Because the probability of error is constant through the execution, during the execution of an arbitrarily deep circuit, a non-recoverable error will eventually happen.

Proven by Aharonov and Ben-Or [12], the Threshold Theorem gives us a framework for arbitrarily long quantum computations.

It proves that when we increase the distance of a code, hence introducing overhead but improving the code capacity, the logical noise rate that we obtain behaves differently depending on the physical error rate that our hardware has:

- If the physical error rate, p is below a certain value, p_{th}, which depends on the correction protocol itself, increasing the distance reduces the logical noise. That is, the benefit of increasing the code capacity is greater than the drawback of increasing the circuit overhead.
- If p is above p_{th}, increasing the distance increases the logical noise. That is, the benefit of increasing the code capacity is smaller than the drawback of increasing the circuit overhead.

The value at which the behavior changes is what we call the Threshold, p_{th}, of a QEC code. We can conclude that, if we have hardware whose physical error rate p is below the threshold p_{th} of a certain code, we can arbitrarily increase the distance of such code to reduce the effective noise. This compensates the increase in the chances of a non-recoverable error (when a circuit depth increases) by increasing the distance of the code.

Note that the threshold, p_{th}, of a given code is not an isolated property of the code. It depends on the actual implementation of the code in a certain hardware. In Chapter 2 we will show how mapping a given code to a given system (a set of qubits with a given inter-connectivity) adds overhead that change the Threshold of the result.

In Chapter 5, we will simulate various QEC codes to visualize this phase transition. By plotting the logical error rate p_L as a function of the physical error rate p for different distances, we will numerically identify the threshold p_{th} as the precise crossing point where scaling the distance inverts its effect on logical noise.

CHAPTER 2

State of the art of QEC codes: Surface codes, color codes and QLDPC codes

2.1 Introduction

In Chapter 1, we established the fundamental principles of quantum error correction (QEC), learning how we can encode logical information into multiple physical qubits to protect it from noise. However, finding the right code for a real-world quantum computer is a delicate balancing act. A good QEC code must offer high fault-tolerance thresholds, require hardware connectivity that is actually physically possible to build, and maintain a reasonable encoding rate so that we do not need a very large number of physical qubits for each logical one we encode.

This chapter explores the evolution of these codes, focusing on three major families: Surface codes, Color codes, and Quantum Low-Density Parity-Check (QLDPC) codes.

2.1.1 About terminology: Topological vs. LDPC Codes

Before analyzing specific codes, we must clarify the terminology we use to classify them:

In modern literature, when researchers say "QLDPC codes", they are often referring colloquially to non-topological QLDPC codes.

- **Topological Codes:** In a topological code, qubits are arranged on a physical lattice (usually in 2D or 3D). Crucially, the stabilizer checks only interact with strictly local, geometrically adjacent qubits. Surface codes and 2D color codes are the quintessential examples of topological codes.
- **Quantum Low-Density Parity-Check (QLDPC) Codes:** A code is considered LDPC if the number of qubits involved in any single parity check (its weight), and the number of checks acting on any single physical qubit, are both strictly bounded by a constant, regardless of the size of the code. Mathematically, surface codes and color codes perfectly fit this definition.
- ***Non-topological* QLDPC Codes:** These codes sacrifice the strict 2D geometric locality in favor of long-range connections. Doing so allows them to break theoretical limits and achieve asymptotically "good" encoding rates (where the number of logical qubits scales linearly with the physical ones).

2.2 Transversality and the Eastin-Knill Theorem

A central goal in Quantum Error Correction (QEC) is executing logical gates fault-tolerantly. A gate is called *transversal* if applying the physical gate independently to each physical data qubit results in the logical version of that gate being applied to the encoded state. Transversal gates are the holy grail of fault-tolerance because they never propagate errors between qubits within the same code block. Because the operations are performed strictly on isolated pairs of physical qubits (qubit-by-qubit), an error cannot spread to other physical qubits within the same logical block. Thus, a single physical error remains a single physical error, which the error-correcting code can safely detect and fix.

However, the **Eastin-Knill Theorem** [13] states a severe theoretical limitation: *No quantum error-correcting code can have a universal set of transversal logical gates*. This means we cannot build a universal quantum computer relying solely on these simple, inherently safe transversal operations.

When a required logical gate is non-transversal, we cannot simply interact the physical qubits of one logical block with another pairwise. Instead, we must rely on complex fault-tolerant protocols that interact deeply with the internal structure and boundaries of the logical qubits. These protocols are challenging to implement and are prone to causing error propagation. Depending on the nature of the target operation, two main techniques are employed:

Multi-Qubit Gates: Lattice Surgery

In Lattice Surgery, merging two logical qubits means activating ancillas between the borders of the two logical qubits and creating two qubit gates between these ancillas and data qubits of the two adjacent logical qubits, effectively creating a chain of stabilizers through the boundary of both qubits. The subsequent measurement of these stabilizer chain is equivalent to measuriring the joint logical operator for both qubits. The act of measuring it is what causes the projection that leaves both logical qubits in an entangled state.

To perform non-transversal operations between two distinct logical qubits (for example, a logical CNOT in codes that do not support it transversally), the standard approach is **Lattice Surgery**. Rather than applying physical gates between individual pairs of qubits across the blocks, lattice surgery temporarily "merges" two distinct logical qubits into one larger logical block by measuring multi-qubit parity checks along the spatial boundary separating them.

After the logical operation is completed, the blocks are "split" again. While this protocol is fault-tolerant in theory, it is practically demanding. It requires executing delicate measurement cycles along the boundaries of the code patches, demanding extra routing and exposing the logical qubits to complex error propagation dynamics during the merging and splitting phases.

Single-Qubit Gates: Magic State Injection and Distillation

To implement non-transversal single-qubit gates—most notably the non-Clifford T-gate, which is essential for universal quantum computation—we use **Magic State Injection** [14]. Instead of applying the gate directly to our data qubit, we prepare an auxiliary logical qubit in a special state called a "magic state". We then entangle our data qubit with this magic state (often using lattice surgery) and measure the auxiliary block, effectively teleporting the non-Clifford operation onto our data.

The critical issue is that preparing a magic state fault-tolerantly directly from physical qubits initially yields a "noisy" state. If we inject a noisy state into our computation, the logical algorithm will fail. Therefore, before injection, these states must be purified using a process called **Magic State Distillation**.

Distillation is a highly iterative and wasteful protocol that consumes several noisy magic states to produce a single, higher-fidelity state. For instance, a standard distillation protocol might consume 15 noisy states to yield just 1 usable state. Because useful

quantum algorithms require large numbers of T-gates, a fault-tolerant quantum computer must continuously manufacture these states in real-time.

To achieve this, quantum processors must dedicate enormous, specialized regions—often referred to as *Magic State Factories*—exclusively to this task. This immense spatial and temporal overhead is precisely what motivates the search for alternative QEC families. For instance, some configurations of color codes [7] inherently support a much richer set of transversal gates, significantly reducing or entirely bypassing the need for extensive magic state distillation.

An example of the distillation overhead is visible in the 2021 article by Gidney & Ekera[15]. In this paper, they propose the factorization of a 2048 bit RSA interger using a quantum processor with 20 million qubits. The article estimates that 25% of these qubtis should be dedicated to Magic State distilation.

2.3 Surface Codes

2.3.1 From 1D Repetition to 2D Topologies

In Chapter 1, we learned that a 1D linear repetition code can only protect against one specific type of error (either bit-flips or phase-flips). Historically, protecting against both required complex concatenation schemes, like Shor's 9-qubit code. The surface code solves this elegantly by expanding into a second spatial dimension. By distributing qubits across a 2D grid, the code can interleave two different types of parity checks simultaneously, monitoring both X and Z errors without the recursive and inefficient overhead of concatenation.

2.3.2 Origins: The Toric Code

This 2D topological approach was famously introduced by Alexei Kitaev in 1997 via the **Toric Code** [6]. It is typically defined on a 2D square lattice where the data qubits reside on the edges. The parity checks (stabilizers) are defined on the vertices (X-checks) and the faces or plaquettes (Z-checks). In this geometric arrangement, every data qubit is continuously monitored by four ancilla qubits: two responsible for X-checks and two for Z-checks. Conversely, every ancilla qubit monitors exactly four adjacent data qubits.

As formulated by Bombín & Martín-Delgado [16], surface codes can also be represented as a lattice with colored faces. In this configuration, data qubits reside on the vertices and stabilizers are defined on the faces, with the color of each face indicating the type of stabilizer. This is known as the rotated surface code representation and is now widely adopted in the literature.

This specific topology is not arbitrary; it is meticulously designed to satisfy the fundamental requirement of stabilizer quantum codes: all parity checks must commute. Because X and Z Pauli operators anticommute on a single qubit, an X-check and a Z-check will only commute if they overlap on an *even* number of shared data qubits. In this lattice geometry, any vertex (X-check) and any adjacent face (Z-check) will always share exactly two data qubits, ensuring that all operators commute globally and the checks can be measured simultaneously without collapsing the logical state.

To avoid the mathematical complexity of handling open boundaries at the edges of the grid, Kitaev imagined a lattice with periodic boundary conditions. In this model, the top edge of the lattice connects to the bottom edge, and the left edge connects to the right edge. Topologically, joining these opposing edges forms a continuous closed surface in the shape of a torus.

2.3.3 Observables, Capacity, and Code Distance

In topological codes, logical operations (logical gates and measurements of the qubit) are not applied to single physical qubits. Instead, logical observables are defined as continuous chains or *strings* of Pauli operators applied sequentially along a path of adjacent data qubits.

A closed string of operations that forms a small, trivial loop on the surface is simply a combination of stabilizers and has no effect on the encoded logical state. However, a string that wraps entirely around the torus—a *non-contractible loop*—fundamentally

changes the logical state. These global, non-contractible strings represent our logical operators (X_L and Z_L). Figure 2.1 shows this.

When visualizing this topology on a 2D grid, these two distinct non-contractible loops correspond to the orthogonal directions of the lattice (East-West and North-South). On a torus, for each of these two directions, two operators, X_L and Z_L, can be applied, effectively codifying 2 qubits (each with 2 operators).

The number of logical qubits k a code can store depends strictly on the topology (the genus) of the surface. A torus has exactly two independent paths to wrap a string (one around the central hole, and one through the tube), meaning the toric code natively encodes exactly $k = 2$ logical qubits, regardless of its physical size.

To increase the protection of the code, we simply increase the density of the grid. By adding more physical qubits, the topological shape (the torus) remains identical, but the length of the shortest non-contractible string grows. This minimum string length is exactly the **code distance** (d). As the grid grows, it takes a longer chain of consecutive physical errors to accidentally form a logical operator, thereby increasing the fault-tolerance while preserving the underlying topological properties.

2.3.4 Planar Implementation and Rotated Surface Codes

While the toric code is mathematically beautiful, manufacturing a quantum processor shaped like a 3D torus is physically impractical, particularly for 2D planar architectures like superconducting circuits.

To implement this on a real chip, we must "cut" the torus and lay it flat. This physical cut transforms the Toric Code into the planar **Surface Code**. However, cutting the torus exposes physical borders. To preserve the ability to correct both X and Z errors, the planar patch must have two types of alternating boundaries: "rough" boundaries (where Z-strings terminate) and "smooth" boundaries (where X-strings terminate). Because the closed loops of the torus are now severed into linear strings that span from one boundary to the opposite, the storage capacity drops: a standard planar patch encodes only $k = 1$ logical qubit.

To further optimize the mapping onto real hardware, the planar surface code is almost universally implemented in its **rotated** version. While this rotated representation perfectly preserves the underlying topological properties, it drastically improves spatial efficiency. Because the code distance d is defined purely by the minimum length of the string connecting opposite boundaries, rotating the orientation of these boundaries by 45 degrees eliminates the wasted corner qubits present in the standard planar cut. Consequently, the rotated surface code requires significantly less physical qubits to achieve the exact same distance d (see Figure 2.1).

2.3.5 Advantages and Disadvantages

The surface code's primary advantage is its incredibly high error threshold (around $\sim 1\%$ for circuit-level noise, see [17]) and its locality, relying on strictly nearest-neighbor 2D grid connectivity. This spatial locality is perfectly aligned with the 2D fabrication constraints of modern planar architectures, making it the industry standard baseline.

However, its main disadvantage is its terrible encoding rate. A standard planar surface code patch encodes exactly one logical qubit ($k = 1$), regardless of how much we scale its distance d. To achieve higher distances and exponentially better protection, the physical footprint grows quadratically ($n \approx d^2$). This massive spatial overhead limits the scalability of surface codes for algorithms requiring thousands of logical qubits.

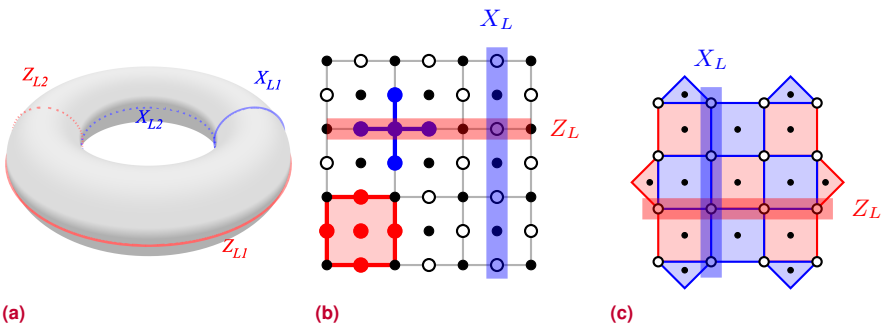


Figure 2.1. *Evolution and representations of the surface code.* ***(a)*** *The toric code representation, illustrating how logical observables correspond to non-contractible continuous strings wrapping around the two fundamental cycles of the torus.* ***(b)*** *The original planar formulation by Kitaev ($d = 4$), with data qubits on the edges and checks on vertices and faces. The red square represents a Z-stabilizer and the blue cross represents a X-stabilizer. A total of 49 physical qubits are needed.* ***(c)*** *The equivalent rotated surface code, placing data on vertices and alternating checks on colored faces to significantly reduce physical overhead. Red faces represent Z-stabilizers and blue faces represent X-stabilizers. The blue and red lines represent the minimum string for the respective operators. In this case, only 31 physical qubits are required. In figures* ***(b)*** *and* ***(c)*** *for a $d = 4$ code, white circles represent data qubits and black circles represent ancilla qubits. The blue and red lines represent the minimum strings for the respective logical operators.*

2.4 Color Codes

2.4.1 Origins and Topologies

Introduced by Bombín and Martin-Delgado [7], topological color codes represent a powerful evolution in the landscape of quantum error correction. They are defined on 2D lattices that strictly satisfy two geometric properties: they must be 3-valent and 3-colorable (typically utilizing red, green, and blue).

A trivalent lattice is a lattice in which each vertex is connected to exactly three edges. A 3-colorable lattice is one where, using only 3 colors, faces can be colored so that two adjacent faces never share the same color.

Unlike the surface code, which physically separates X-checks and Z-checks by placing them on vertices and faces respectively, color codes place both X and Z checks on the same faces of the lattice, while data qubits reside exclusively on the vertices.

This overlapping check structure is mathematically robust due to the inherent geometry of these lattices. In any 3-valent, 3-colorable lattice, every face is guaranteed to have an even number of vertices. Consequently, an X-check and a Z-check defined on the exact same face will overlap on an even number of data qubits, guaranteeing that they commute. Furthermore, any two adjacent faces will always share exactly one edge (which contains exactly two vertices). Thus, an X-check on one face will always share two data qubits with a Z-check on an adjacent face, ensuring global commutativity across the entire code block, as required by the Stabilizer Formalism presented in Section 1.4.

2.4.2 2D Color Code Configurations

Because of the strict mathematical requirement that the lattice must be 3-valent and 3-colorable, there is only a finite number of uniform tilings (Archimedean tilings) in the 2D plane that can support a color code, see [18]. Specifically, there are exactly three configurations, shown in Figure 2.2:

While there are only three uniform tilings, an infinite number of 3-valent, 3-coloreable lattices can be defined if uniformity is not required. These are rarely studied in the literature because they are more difficult to scale and fit on a quantum chip with a regular pattern and because their error correction properties are less predictable.

- **6.6.6 Lattice:** A regular lattice composed entirely of hexagons.

- **4.8.8 Lattice:** A semi-regular lattice where each vertex connects a square and two octagons.
- **4.6.12 Lattice:** A more complex semi-regular lattice composed of squares, hexagons, and dodecagons. While theoretically interesting, its extremely high-weight stabilizers (weight-12) make it practically nonviable for current hardware.

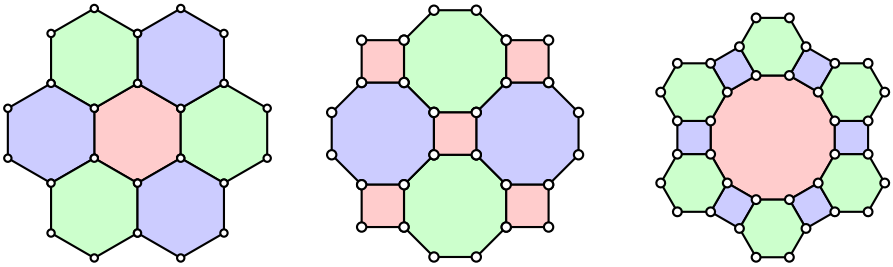


Figure 2.2. *The three valid 2D configurations for topological color codes: the 6.6.6 hexagonal lattice, the 4.8.8 square-octagonal lattice, and the 4.6.12 lattice. The faces are 3-colored (e.g., red, green, blue) such that no two adjacent faces share the same color. Data qubits are represented with white circles. There are two ancilla qubits in each colored face, which are not represented in the image.*

The $4.8.8$ lattice presents unique properties that make it particularly interesting, which motivates our decision of performing our simulations with this lattice:

- Lower qubit requirements: For the 4.8.8, 6.6.6, and 4.6.12 color code geometries, the number of data qubits n as a function of the distance d is given by $n_{4.8.8} = \frac{d^2+2d-1}{2}$, $n_{6.6.6} = \frac{3d^2+1}{4}$, and $n_{4.6.12} = \frac{3d^2-6d+5}{2}$, respectively, with the 4.8.8 lattice being the one with best asymptotical ratio.
- As a doubly-even code, the 4.8.8 lattice enables a transversal implementation of the full Clifford group, including H, $S^\dagger$ and CNOT. While all 2D color code lattices allow for transversal Clifford group gates, only the 4.8.8 geometry allows the S gate to be implemented not only transversally, but uniformly across all physical qubits. Lattices containing faces whose number of sides is not a multiple of four (such as the hexagons in the 6.6.6 and 4.6.12 codes) require the transversal operation to alternate between S and $S^\dagger$ gates.

2.4.3 Topological Observables: Colored Strings and Nets

Just as in the surface code, logical observables in color codes are formed by continuous paths of Pauli operators acting on the data qubits. However, the 3-colorability of the lattice introduces a difference, as two qubits of the string can be connected through an edge or through a face.

In a color code, strings are *colored*. A string can only be a border between faces of the other two colors, but it can go through faces of its color. For example, a “red” string of Pauli operators will strictly run along edges that separate green and blue faces, and will cross red faces.

This color property leads to a unique topological feature: **string branching**. A string of one color can “unfold” or branch into two strings of the remaining colors at any vertex. For instance, a red string can split into a green string and a blue string, forming a structure known as a *net*. This topological flexibility means that logical operators do not

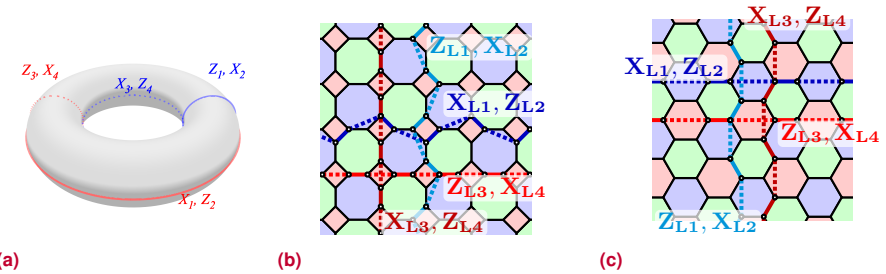


Figure 2.3. *Logical observables in a toric color code.* ***(a)*** *Logical observables correspond to color-independent non-contractible continuous strings wrapping around the two fundamental cycles of the torus. Color codes allow us to codify two logical operators in the same string, with the logical observables for each qubit being orthogonal.* ***(b)*** *The 4 independent strings in a* $4.8.8$ *lattice.* ***(c)*** *The 4 independent strings in a* $6.6.6$ *lattice.*

have to be simple linear loops; they can be complex, branching networks of operators that span the lattice.

These strings and nets define the logical operators. When two non-contractible nets of X and Z operators intersect at an odd number of data qubits, they anticommute, forming a valid logical qubit. Because of this rich combinatorial structure, a color code is twice as dense as a surface code. While a surface code wrapped on a torus natively encodes $k = 2$ logical qubits, a color code on the exact same toroidal surface encodes $k = 4$ logical qubits (see Figure 2.3). For planar implementations, triangular patches are typically used, which encode $k = 1$ logical qubit but offer remarkably different boundary symmetries compared to the square patches of surface codes (see Figure 2.4).

Despite the lattice being three-colorable and supporting strings of three distinct colors, only two of these colors are strictly independent from an error-correction perspective. In a 2D color code, every data qubit is located at a vertex shared by exactly three faces—one red, one green, and one blue. Consequently, any single error will always trigger a defect in exactly one face of each color. Because these syndrome defects are inherently created in triplets, the parity information gathered from any two colors fully determines the state of the third.

2.4.4 Advantages: Transversality and 3D Topologies

Color codes offer massive theoretical advantages over surface codes, primarily regarding logical gate execution. As discussed earlier in this chapter, finding transversal gates is highly desirable to prevent error propagation.

Color codes natively possess a fully transversal Clifford gate set, meaning that critical operations like the Phase gate (S), Hadamard (H), and CNOT can be applied strictly pairwise, without the need for complex and noisy lattice surgery.

Furthermore, the color code topology can be generalized into three dimensions (3D). In 3D configurations, certain color codes allow the transversal execution of non-Clifford gates, such as the T-gate. This is a very significant advantage, as it bypasses the need for Magic State Distillation factories required by the surface code.

Because of the Eastin-Knill Theorem, no QEC code can provide a universal set of transversal gates. Consequently, 3D color codes must sacrifice the transversal H gate (which are easier to distill) in order to support a transversal T gate. To circumvent this and achieve full universality without distillation, techniques like dimensional jumping are employed. This allows the code to transition between 2D and 3D topologies, applying transversal H gates while in the 2D configuration.

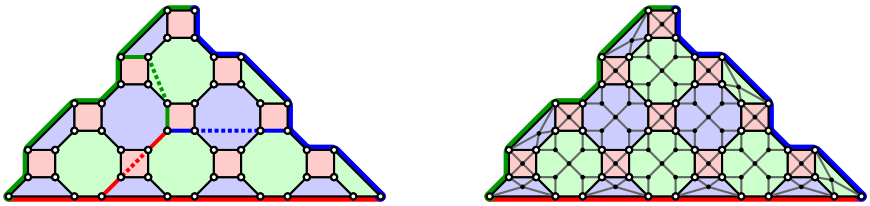
2.4.5 Disadvantages and Processor Mapping

Despite their theoretical superiority, 2D color codes face significant physical implementation hurdles. Their circuit-level threshold is generally lower than that of the surface

code [18], meaning they require underlying hardware with significantly lower baseline noise to begin correcting errors effectively.

Moreover, the stabilizers acting on the faces involve a high number of qubits. For instance, in the 4.8.8 lattice, the octagonal faces represent weight-8 parity checks. Measuring a weight-8 stabilizer dynamically without exposing the data to correlated errors is a mapping challenge on modern hardware, which typically features restricted 2D grid connectivity.

Implementing these codes on real processors requires highly optimized, specialized layouts. For the **4.8.8 lattice**, Fowler [19] demonstrated that it is possible to map this topology onto a standard 2D square architecture, although it requires replacing the ancilla qubit in octogonal faces with 5 ancilla qubits, 4 of them connected to two data qubits each, and the fifth one connected to the 4 ancilla qubits ((see Figure 2.4)).



(a) (b)

Figure 2.4. *Planar topology of a 4.8.8 lattice color code.* ***(a)*** *Definition of the triangular patch for a planar topology, in which we must use string-nets to define the logical observables. Each string-net can encode the logical X and Z operators in the color code, but the individual strings must terminate at a boundary of their own color, which inherently requires branching.* ***(b)*** *Mapping of the color code patch to a quantum processor with a square grid of qubits. For the octagonal faces, we must use 5 ancillas to perform the stabilizer measurements. Ancilla qubits in the tetrahedrons that appear in the boundaries do not map the square grid and are intentionally simplified in this diagram.*

Similarly, the **6.6.6 hexagonal lattice** maps naturally to certain architectures, but it is still a non-trivial problem. Recent publications by Lacroix et al. [20] outline an optimized mapping for the 6.6.6 code.

2.5 Non-topological QLDPC Codes

The BPT bound mathematically proves that for any quantum code restricted to local interactions in 2D, the parameters must satisfy $kd^2 \leq O(n)$. This implies that the physical overhead per logical qubit (n/k) must grow at least as d^2. If higher error protection is needed (higher d), the physical footprint explodes quadratically.

To understand the massive savings: a typical BBC can encode 12 logical qubits with a distance of $d = 12$ using just 144 physical qubits in total. To achieve the same parameters ($k = 12, d = 12$) using standard surface codes, one would need $12 \times 12^2 = 1728$ physical qubits. This represents more than a $10\times$ reduction in hardware footprint.

While 2D topological codes are very hardware-friendly, their reliance on local 2D planes strictly limits their scalability. The fundamental bottleneck of local 2D codes is governed by the Bravyi-Poulin-Terhal (BPT) bound [21]. To build future quantum supercomputers with millions of qubits and increasing distances for longer computations, this geometric restriction imposes a prohibitive physical overhead.

This fact has drawn massive attention to non-topological Quantum Low-Density Parity-Check (QLDPC) codes. By sacrificing strictly local 2D connectivity and embracing long-range physical connections, these codes break the BPT bound. In a topological code, increasing the distance requires making the 2D patch larger, which squares the number of physical qubits. QLDPC codes, instead, can maintain a constant encoding rate while the distance grows linearly with the number of physical qubits. This removes the quadratic penalty for fault tolerance.

2.5.1 Bivariate Bicycle Codes (BBC)

A prominent recent breakthrough in the practical domain is the Bivariate Bicycle Code

family [22]. These codes prove that we can encode dozens of logical qubits with high distances using only a fraction of the physical qubits required by topological codes.

The core ingenuity of BBCs lies in how they guarantee that X-type and Z-type stabilizers commute, which is the most difficult challenge when designing any quantum code. In the stabilizer formalism, an X-stabilizer and a Z-stabilizer commute if and only if they overlap on an even number of physical qubits.

Instead of trying to manually guess which qubits to connect to satisfy this rule, BBCs use a clever mathematical trick. The physical qubits are divided into two equal groups (the "bicycles"). Then, the designers use a polynomial with two variables, x and y, to define which qubits are grouped to form a stabilizer.

Physically, the variables x and y simply represent geometric shifts: x acts as an instruction to "shift one position to the right" and y means "shift one position down" on a virtual grid. A polynomial like $1 + x + y^2$ acts as a wiring instruction that tells the hardware: "to build this stabilizer, apply the operator to a given qubit (1), the qubit to its right (x), and the qubit two steps down (y^2)".

Because polynomials naturally commute in mathematics (multiplying $P_1 \times P_2$ is identical to $P_2 \times P_1$), generating the stabilizer overlapping rules from them automatically guarantees that any X-stabilizer and Z-stabilizer will always intersect on an even number of qubits. Furthermore, by keeping these "wiring instructions" very short (low degree polynomials), the number of physical qubits involved in each stabilizer remains very low (Low-Density), which is essential to prevent error propagation during syndrome measurements.

Their physical implementation, however, is still an open engineering challenge. Because the shifts dictated by the polynomials can connect qubits that are far apart on the grid, these codes require high-fidelity, long-range couplers. Current promising proposals to realize this non-local connectivity include dynamic optical interconnects or neutral atoms that are physicaly moved with optical tweezers. Therefore, non-topological QLDPC codes represent the ultimate frontier for next-generation fault-tolerant architectures.

CHAPTER 3

Modeling quantum error

In order to design, evaluate, and compare Quantum Error Correction (QEC) protocols, we must first establish a framework to describe how errors occur in quantum systems. A decoder cannot be effectively designed or benchmarked without a clear definition of the noise it is expected to correct.

Simulating continuous noise for large-scale systems is computationally intractable. Fortunately, as discussed in Section 1.2, quantum errors can be discretized into a finite set of Pauli errors (X, Y, and Z). Therefore, in the context of QEC simulation, we can safely model noise as discrete probabilistic Pauli operations.

3.1 Types of quantum noise

Before deciding where errors occur in a circuit, we must define the probability distribution of the errors themselves. The choice of noise type drastically affects the performance of both the QEC code and the decoder.

3.1.1 Depolarizing noise

The depolarizing channel is the most standard noise model used in QEC literature due to its symmetry and mathematical simplicity. Under this model, a qubit has a probability p of suffering an error, and when an error occurs, it is equally likely to be an X (bit-flip), Z (phase-flip), or Y (both) error. Thus, each specific Pauli error occurs with probability $p/3$.

The depolarizing channel is widely used for benchmarking decoders because it provides a "worst-case" scenario of symmetric uncertainty. If a decoder performs well under depolarizing noise, it is generally robust across different hardware platforms.

3.1.2 Biased noise

Real quantum hardware rarely exhibits perfectly symmetric noise. In many physical qubit implementations, such as superconducting transmons, the quantum state is far more susceptible to losing its phase information than to losing its energy. This results in **biased noise**, where one type of error (typically Z phase-flips) is significantly more probable than the others.

When the noise is highly biased, standard CSS codes may not be optimal. Instead,

The XZZX surface code has stabilizers that combine Z and X detectors and is therefore not a CSS code.

tailored codes (like the XZZX Surface Code) or tailored decoders can exploit this asymmetry by assigning lower weights to Z errors, vastly improving the threshold for that specific hardware.

3.1.3 Correlated noise

Standard simulations usually assume that errors are independent and identically distributed, meaning the probability of an error on one qubit does not depend on the state of its neighbors. However, physical systems suffer from **correlated noise**, such as cosmic ray impacts or crosstalk between neighboring microwave lines, which can cause simultaneous errors on multiple adjacent qubits. While more difficult to model computationally, analyzing correlated noise is critical because multi-qubit errors can easily form logical strings that bypass the topological protection of the code.

3.2 Error Models

While the *type* of noise defines the probability distribution of the Pauli operators, the *error model* defines the abstraction level of the circuit where these errors are injected. We classify error models into three hierarchical categories: code-capacity, phenomenological, and circuit-level noise.

3.2.1 Code-capacity noise

While physically unrealistic, the code-capacity model is an essential theoretical tool. It isolates the raw error-correcting capability of the code's geometry and the base performance of the spatial decoder, without the confounding effects of faulty syndrome extraction circuits.

The **code-capacity model** is the simplest and most abstract error model. It assumes that all quantum gates, measurements, and state preparations are perfectly flawless. Errors are only injected into the data qubits while they are "idle".

Under this model, the syndrome extracted by the ancilla qubits is always 100% reliable. Because measurements never fail, the decoder only needs to solve a 2D spatial problem (matching the errors on the lattice).

3.2.2 Phenomenological noise

In reality, the classical measurement hardware and the ancilla qubits used to extract the syndrome are also susceptible to noise. The **phenomenological noise model** builds upon the code-capacity model by adding **readout errors**.

In this model, data qubits suffer errors with probability p, and the classical syndrome bits obtained from the measurements are independently flipped with probability q (usually $p = q$). However, the entangling gates used to compute the parity checks are still assumed to be perfect.

Because the syndrome itself can now lie, the decoder can no longer trust a single measurement round. A faulty measurement could trick the decoder into applying an incorrect correction, creating a logical error. To solve this, the syndrome extraction must be repeated over time (typically d rounds for a distance-d code). The decoder must now resolve a 3D spatial-temporal graph, tracking how detection events appear and disappear across time steps.

*Circuit-level noise introduces a critical challenge: **error propagation**. A CNOT gate will propagate an X error from its control to its target, and a Z error from its target to its control. A single physical fault occurring in the middle of a syndrome extraction routine can propagate into a multi-qubit error on the data qubits (known as a "hook error").*

3.2.3 Circuit-level noise

The **circuit-level noise model** is the most realistic and demanding simulation framework. It removes all assumptions of perfection. In this model, noise is injected into every physical operation:

- **Initialization**: Qubits may be prepared in the wrong state.

- **Idle operations**: Qubits suffer decoherence while waiting for other operations to finish.
- **Single-qubit gates**: A depolarizing channel is applied after every single-qubit rotation.
- **Two-qubit gates**: A two-qubit depolarizing channel is applied after every entangling gate (e.g., CNOT).
- **Measurements**: Measurement outcomes can be flipped, as in the phenomenological model.

Because of error propagation, circuit-level noise can generate correlated errors on the data lattice from a single independent fault. Decoders operating under circuit-level noise must be explicitly aware of the circuit scheduling (the specific order of the CNOT gates) to correctly anticipate and decode these correlated hook errors.

3.3 Relationship between the models

These three error models form a strict hierarchy. Circuit-level noise is the superset that encapsulates all physical reality. If we take a circuit-level simulation and artificially set the error rates of all gates and idle cycles to zero (leaving only measurement errors and a baseline data qubit error), we recover the phenomenological model. If we subsequently set the measurement error rate to zero, we are left with the code-capacity model.

We use the $[p_{data} : p_{meas} : p_{gate}]$ notation to define the probability ratios between the three models. As an example, $[1 : 0.5 : 0]$ represents a phenomenological noise mode with data errors happening twice as much as measurement errors.

Throughout this thesis, we will strictly simulate non-correlated depolarizing noise. We will utilize the code-capacity model to analyze the fundamental topological properties of Color Codes, and deploy the circuit-level noise model when benchmarking their performance against realistic quantum hardware constraints. The exception will be the initial implementation in Chapter 5, in which we will use different noise models to illustrate its impact and various properties of the QEC codes.

CHAPTER 4

QEC decoders for color codes

Having established the foundational principles of quantum error correction, and having explored both the structure and characteristics of color codes and the physical reality of error models, it is now time to address the final critical component of the QEC cycle: the decoders. A decoder is the classical algorithm responsible for transforming a set of syndrome measurements into a concrete diagnosis. In other words, it formulates a prediction of the error that has affected the logical observable, or equivalently, it infers the correct logical observable given the faulty measured one. The execution speed and accuracy of the decoders are major bottlenecks in realizing large-scale fault-tolerant quantum computation, and there's often a trade-off between them. Speed is relevant because the decoding process must ideally keep pace with the quantum circuit operation, as sometimes errors must be corrected during the execution. However, to increase decoding speed, accuracy may be sacrificed.

To achieve high accuracy, a decoder must do more than just find any arbitrary error string that satisfies the observed syndrome; it must ideally identify the most probable error chain. At this point, we must introduce the concept of the *Decoding Graph* or **Detector Error Model**, DEM. The DEM is a structured representation that captures the specific probabilities of the different error mechanisms that trigger detection events in the quantum circuit. By incorporating a DEM, decoders can leverage these probability distributions to make informed decisions. This additional statistical context allows the decoder to determine the actual error chain with a significantly higher success rate.

*The Decoder Error Model is often referred to by different names in the literature: Syndrome Graph, Decoding Graph or Matching Graph. Because our focus is in QEC simulation, we will prefer the term **Detector Error Model** or DEM, which is the term used by `stim`. The DEM acts as a bridge between the physical noise phenomena (such as gate infidelities or readout errors) and the abstract graph or hypergraph structures used by most decoding algorithms to find the optimal correction.*

In this chapter, we will not dive into the programming aspects or the specific implementation details of decoders. The practical implementation of a basic decoder will be thoroughly covered in Chapter 5, where it will also be compared against the standard Minimum Weight Perfect Matching (MWPM) algorithm to explicitly demonstrate the clear advantage of incorporating the DEM into the decoding process.

4.1 Minimum Weight Perfect Matching (MWPM)

The Minimum Weight Perfect Matching (MWPM) was the first algorithm to demonstrate that topological codes could be practically decoded. Its theoretical mapping to the surface code was established by Dennis, Kitaev, Landahl, and Preskill in 2002 [23].

*MWPM relies in two fundamental things: **Perfect Matching**, meaning that it matches pairs of nodes with a set of edges, so no node is left unpaired and no node belongs to multiple matches, and **Minimum Weight**, meaning that the total weight of the selected set of edges is the minimum possible to perfectly match the active nodes.*

MWPM is arguably the most famous and widely used decoder in the field of quantum error correction. Originally conceived as a solution to a classical optimization problem by Jack Edmonds in 1965 [24], MWPM is designed to find a perfect matching in a weighted graph such that the sum of the weights of the edges in the matching is minimized.

In the context of QEC, and specifically for the Surface Code, the decoding problem maps naturally to a standard graph problem. When a single Pauli error occurs on a data qubit in a surface code, it typically flips the measurement outcome of exactly two adjacent stabilizers. These two activated stabilizers are the "defects" or "detection events" that form the syndrome. The decoding process consists of constructing a **syndrome graph** where:

- **Nodes** represent the activated stabilizers.
- **Edges** represent the possible physical errors (or chains of errors) that could have caused those stabilizers to flip.
- **Weights** on the edges are assigned based on the error probabilities. Lower probability errors are assigned higher weights.

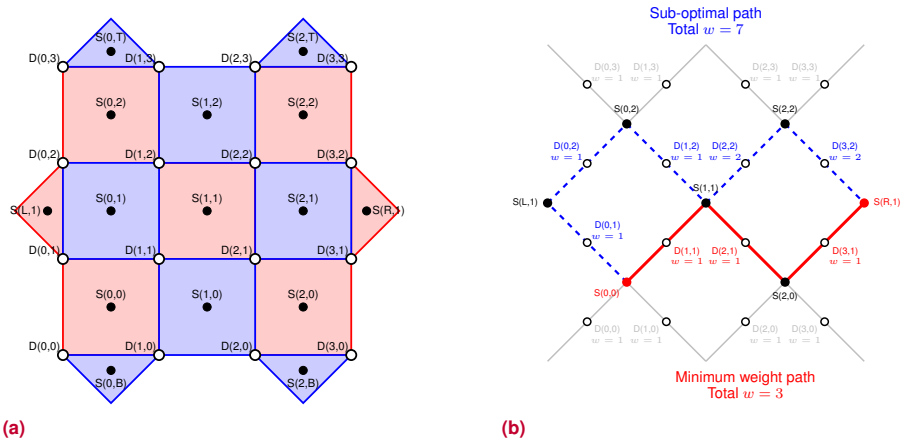


Figure 4.1. *A visual representation of the Minimum Weight Perfect Matching problem under a code-capacity noise model.* ***(a)*** *The rotated surface code lattice introduced in Chapter 2. Data qubits are labeled as $D(x, y)$ and ancillas (stabilizers) as $S(x, y)$.* ***(b)*** *The full syndrome matching graph specifically for X-type errors, consisting of 7 nodes (the Z-stabilizer ancillas, shown as small black circles) and 16 edges (one for each physical data qubit–each possible X-type error in the code-capacity model–, represented by white circles indicating the weight (likeliness) of this error. Two possible error chains are highlighted, one of them being the one with minimum weight.*

The MWPM algorithm efficiently searches this graph to find the pairings of defects that minimize the total weight. This is illustrated in Figure 4.1b. Once the optimal pairing is found, the decoder identifies the most probable error chain and applies the corresponding correction. This approach is highly effective and its implementation ([25]) serves as the benchmark against which almost all new decoders are measured.

4.1.1 Edmonds' Blossom algorithm

To understand how MWPM achieves this globally optimal pairing, it is necessary to look at its underlying mathematical engine: Edmonds' Blossom algorithm. A naive greedy approach—simply pairing the closest defects as soon as they are found—can easily get trapped in local minima, leaving isolated defects that force very high-weight connections

later. Edmonds' algorithm systematically explores the graph to guarantee the absolute minimum weight solution.

This exploration is intuitively visualized using expanding regions. The algorithm begins by simultaneously growing a "bubble" outwards from every active defect. These bubbles expand across the adjacent data qubits in the graph, to other non-active defects. When the boundaries of two expanding bubbles collide, the two defects are temporarily matched. The algorithm assumes that the path followed by the bubbles corresponds to the chain of physical errors that connected them.

As the bubbles continue to grow, they often encounter defects that are already matched to someone else, creating topological conflicts. The algorithm efficiently resolves these conflicts by re-evaluating its temporary matchings as new connections are discovered. Conflicting matches are resolved using *alternating paths*—routes that alternate between currently assumed errors (matched edges) and assumed healthy qubits (unmatched edges). If an alternating path is found connecting two free defects, the algorithm "flips" the entire path's status, simultaneously correcting past suboptimal assumptions and increasing the total number of matched pairs.

Let's see an example. Defects A and B are matched ($A \Rightarrow B$) and then defect C's expanding bubble reaches B. The path between them is stored as a non-matched path ($C \rightarrow B$). Defect A is re-activated and it's bubble begins growing again. If the bubble reaches an active unmatched defect D, then an alternating path forms ($D \rightarrow A \Rightarrow B \rightarrow C$). This path has two non-matched segments and one matched segment. The algorithm "flips" the entire path's status ($D \Rightarrow A \rightarrow B \Rightarrow C$), correcting past suboptimal assumptions and increasing the total number of matched pairs.

Edmonds' algorithm succeeds where other algorithms fail because of how it handles cycling paths: the temporary connections form a closed loop consisting of an odd number of nodes. Odd-length cycles can't be paired internally (the flip will not fix the alternating chain). The Blossom algorithm overcomes this by temporarily collapsing the entire odd-cycle into a single "super-node" or *blossom*. This new blossom then resumes expanding as a single entity to find an external pair. Once all defects in the graph are paired, the algorithm unfolds the blossoms back into their original nodes and resolves the internal pairings. This continuous process of growing bubbles, shrinking odd-cycles, and unfolding them guarantees finding the most probable physical error chain.

The Challenge of MWPM in Color Codes

While MWPM is exceptionally well-suited for Surface Codes, it cannot be directly applied to 2D Color Codes. The reason lies in the fundamental topology of the code.

In a surface code, a physical error creates a 1-dimensional string connecting two defects. However, in a color code, a single physical Pauli error activates **three** adjacent stabilizers. Consequently, an error chain in a color code is not a simple path between two points, but a branching, 2-dimensional structure that connects multiple groups of three defects.

In the following sections we will study different alternatives to decode color codes, some of which heavily rely on MWPM.

4.2 Union-find

While the Minimum Weight Perfect Matching (MWPM) algorithm sets the standard for decoding accuracy, its worst-case computational complexity does not scale well with the number of nodes in the syndrome graph. Decoders must operate fast enough to prevent a backlog of uncorrected errors during the circuit execution, wich causes a bottleneck as fault-tolerant architectures scale to thousands of physical qubits. To address this speed bottleneck, Delfosse and Nickerson introduced the Union-find decoder in 2018 [26], achieving an almost-linear worst-case complexity while maintaining a decoding threshold highly comparable to MWPM.

The overall worst-case complexity is bounded by $O(V\alpha(V))$, where V is the number of vertices and $\alpha(V)$ is the inverse Ackermann function—a function that grows so slowly that it evaluates to less than 5 for any practically conceivable graph size.

The primary innovation of the Union-find decoder is the replacement of the global path optimization of Edmonds' Blossom algorithm with a greedy solution. As mentioned earlier, a naive greedy approach would easily fall into local minima. Union-find circumvents this by proposing a greedy cluster-growth strategy instead. Similar to MWPM, bubbles

grow and merge upon collision, but rather than evaluating the optimal pairing when two bubbles collide, the algorithm simply continues growing them as long as they contain an odd number of defects. Once every cluster contains an even number of defects (meaning all defects are safely encapsulated), a secondary phase acts strictly within each cluster to definitively pair the defects. The algorithm thus operates in two strictly sequential phases:

*The **Disjoint-Set** data structure tracks elements partitioned into disjoint subsets, supporting two efficient operations: **Find** identifies the root of a subset (the unique element of the subset that identifies this subset), and **Union** merges two subsets. Checking or merging clusters is executed in practically constant time, bringing the computational cost of the algorithm to near-linear with respect to the number of nodes.*

At an intersection, if both (or neither) of the child branches bring an unpaired defect, the error strings are combined and the upwards node is left unmarked. If only one of the two branches brings an unpaired defect, then the upwards node is marked as an unpaired defect so the pruning can continue upwards.

1. **Cluster Growth:** Initially, every individual defect is placed in its own separate cluster. Using the standard *Disjoint-Set* (or Union-find) data structure, the algorithm tracks these disjoint clusters. A cluster is classified as *valid* if it contains an even number of defects, or if it touches a boundary. If a cluster is valid, its growth pauses. Conversely, any *invalid* (odd-parity) cluster grows uniformly by absorbing its neighboring edges. When two growing clusters collide, the algorithm executes a `union` operation to merge them into a single, larger cluster. This iterative growth and merging process continues until every cluster in the graph becomes valid. As opposed to MWPM's blossom contraction, the Disjoint-Set data structure allows an easier handling of a cluster colliding with itself (creating an internal loop) thanks to the *find* operator.
2. **Path Resolution:** Once all clusters are validated, the algorithm must decide which specific edges inside each cluster correspond to the physical errors. For each valid cluster, the Union-Find process has already defined a tree (i.e. a graph without loops). This allows applying a highly efficient *peeling algorithm*: starting from the leaf nodes of the tree and working toward the root, the algorithm greedily assigns error correction strings to pair all internal defects. Each leaf node is evaluated and then pruned. If the node is a defect, the pruned edge is added to an error correction string and the next upwards node is marked as an unpaired defect to continue the pruning from it. When two branches intersect, the pruning on that path pauses until both branches reach the intersection point.

The algorithm described so far does not account for different error probabilities. Because of that, it can lead to lower accuracy than MWPM. *Weighted* variants of Union-find incorporate the DEM edge weights directly into the growth phase, causing clusters to grow faster through edges with higher probabilities (lower weights). This bridges the accuracy gap with MWPM while retaining fast execution speeds.

Application to Color Codes

Like MWPM, the standard Union-find decoder is fundamentally a graph-based algorithm designed to pair defects connected by 1-dimensional strings. Consequently, applying Union-find directly to the branching hypergraphs of 2D Color Codes presents the exact same topological challenge outlined in Section 4.1. To decode color codes efficiently, Union-find is typically integrated as the fast computational engine within reduction frameworks, such as Projection or Restriction decoders, which map the complex hypergraph onto standard surface code sub-graphs.

CHAPTER 5

Implementation and simulation of QEC codes: Framework setup and baseline models

We have so far conducted a profound theoretical study of the different QEC codes, decoders, and error models. Moving from theory to practice requires specialized computational tools capable of simulating quantum systems efficiently.

Simulating arbitrary quantum circuits is exponentially hard for classical computers; however, the Gottesman-Knill theorem [27] guarantees that circuits restricted to Clifford group operations (which include stabilizer measurements, CNOTs, and Pauli gates) can be simulated efficiently in polynomial time.

The Gottesman-Knill theorem is referred to in the original Gottesman paper simply as Knill's theorem, as it was initially only privately communicated. It is now commonly known as the Gottesman-Knill theorem to recognize both Knill's original insight and the subsequent formal proof by Gottesman using his Stabilizer formalism.

In this chapter, we establish our experimental framework using `stim` [8], a high-performance Clifford simulator that relies on the referred Gottesman-Knill theorem and its subsequent improvements by Aaronson and Gottesman[28], and incorporates some additional improvements. We will build our simulations progressively: starting with a basic repetition code to understand the mechanics of syndrome extraction and decoding, moving through different noise models, and finally implementing the 9-qubit Shor code to empirically demonstrate the Threshold Theorem.

5.1 Simulating QEC codes and decoders with `stim`

Stim is a QEC specialized simulator. The primary advantage of `stim` is its ability to sample detection events –syndromes– and logical observables –flips in the logical state of our code– extremely fast. Instead of tracking the full state vector, it tracks how Pauli errors propagate through the circuit. To utilize this tool, a QEC simulation requires three main components: a circuit defining the code structure, a noise model that injects errors, and a classical decoder that interprets the syndromes.

5.1.1 Simulating the repetition code under code-capacity noise

To introduce the simulation pipeline, we begin with the simplest QEC protocol: the distance-d repetition code. As discussed in Chapter 1, this code protects against a

single type of error (e.g., bit-flips) by encoding one logical qubit into d physical data qubits, requiring $d-1$ additional ancilla qubits to perform parity measurements.

For our baseline simulation, we use a **code-capacity noise model** (see Chapter 3) using a depolarizing channel.

Stim makes it exceptionally easy to implement a basic repetition code. The building blocks of this simulation are:

- **The QEC code circuit**. `stim`'s library already provides a method to easily generate a basic repetition code. This generates not only the circuit for the encoding, but the necessary ancilla qubits and measurements for syndrome extraction.
- **Injecting the error**. As part of building the circuit, `stim` allows inputing the physical error rates for different types of noise. In this case, to simulate code-capacity noise, we use the parameter `before_round_data_depolarization`.
- **Decoding the error**. The `pymatching` library [25] will allow us to decode the error using the MWPM algorithm that we discussed in Chapter 4. For such decoding to happen, `stim` requires the circuit to have specific tags appended (`DETECTORS`) to specific measures. In this example, the generated circuit already contains them.

The code in Listing 5.1 illustrates how this simulation can be performed. It is important to understand how `stim` works. The `sample` method returns two things:

1. A binary array of detection events (in the code, `syndromes`). It contains one value for each measured detector. The value is 0 if the detector observed the expected parity, and 1 if the detector was activated, indicating the presence of an error.
2. A binary array of logical outcomes (in the code, `actual_observables`). Rather than representing logical states, these values act as boolean flags. A value of 1 indicates that the accumulated physical errors during that shot successfully flipped the corresponding logical observable.

The preparation of the initial state of a circuit is not generally required. However, noise models can be asymmetric (e.g. a qubit may be more likely to flip from $|1\rangle$ to $|0\rangle$ than from $|0\rangle$ to $|1\rangle$) and so can be QEC codes (the 9-qubit Shor code has better protection against bit-flip errors than against phase-flip errors). In these cases, preparing the initial state may be required.

During the simulation, errors can occur with a certain probability p. When they happen, they are tracked and cascade down to the detectors and observables, allowing `stim` to determine both if a detector is triggered and if an observable flips. If the code and the decoding succeed, both should match. Because flips, rather than states, are tracked, the circuit does not have to be prepared on a certain state, we only evaluate its capacity to preserve its state.

Figure 5.1 shows the logical error rate as a function of the physical error rate for this code with two different decoders. Note that, as a basic repetition code, this code only corrects for bit-flip errors.

Source: `src/chapter_5/s1_stim_rep_code.py`

```python
def sample_repetition_code_mwpm(distance: int = 3, shots: int = 10000) -> None:
    """
    Simulates a repetition code over a range of physical error rates,
    using the pymatching MWPM decoder.
    """
    physical_error_rates = np.linspace(0.01, 0.15, 10)
    logical_error_rates = []

    for p in physical_error_rates:
        # Define the circuit
        circuit = stim.Circuit.generated(
            "repetition_code:memory",
            distance=distance,
            rounds=1,
            before_round_data_depolarization=p
        )

        # Sample the circuit to get syndromes and actual observables
        sampler = circuit.compile_detector_sampler()
        syndromes, actual_observables = sampler.sample(shots, separate_observables=True)

        # Decode the syndromes using PyMatching
        matcher = pymatching.Matching.from_stim_circuit(circuit)
        predicted_observables = matcher.decode_batch(syndromes)

        # Calculate the logical error rate as the cases where the
        # actual observables differ from the predicted ones
        num_errors = np.sum(predicted_observables != actual_observables)
        logical_error_rates.append(num_errors / shots)

    return physical_error_rates, logical_error_rates
```

Listing 5.1. *Simulation 1: Basic repetition code simulation*

5.1.2 Decoders: From Minimum Weight Perfect Matching to Custom Logic

Once the circuit is executed and noise is applied, `stim` outputs an array of detection events. The role of the decoder is to map these events to a predicted logical error.

As seen in Chapter 4, the standard approach for 1D repetition codes and 2D surface codes is the **Minimum Weight Perfect Matching (MWPM)** algorithm. In our framework, we implement this using the `PyMatching` library.

To better understand the internal workings of a decoder, we implement a custom **Majority Vote Decoder**. Unlike the graph-based approach of MWPM, this decoder resolves errors by directly evaluating the local parity checks and "voting" on the most likely state of the data qubits.

The code Listing 5.2 shows a very simple implementation of such decoder, which can be used with the same repetition circuit created in Listing 5.1.

Note that we show here the specific part of the code that decodes a single shot of the sampling. When the circuit is simulated, multiple shots are run to get statistically significant information

Source: `src/chapter_5/s2_majority_vote_decoder.py`

```python
# We iterate over each observable, which corresponds to a block of detectors
# This would allow codes encoding multiple logical qubits
for i in range(num_observables):
    # The number of detectors in our code, for a given observable
    # is a function of the code distance.
    # We may receive additional detectors (frontier detectors) that we can ignore.
    # For the 3-qubit repetition code, we have 2 detectors per observable.
    # We need to iterate over the blocks of detectors for each observable
    start_index = i * detectors_per_observable
    end_index = start_index + (distance -1)
    local_syndrome = shot_syndrome[start_index:end_index]

    # For a repetition code, decoders are based on pairs of data qubits
    # that are chained, e.g. (q0, q1) and (q1, q2).
    # We achor to the first qubit, assume that it has not flipped (set to 0).
    qubits_state = [0]
    current_state = 0

    # Let's iterate over the detectors
    for detector_value in local_syndrome:
        # If a given detector is triggered, it indicates a flip
        # of this qubit relative to the previous one.
        if detector_value == 1:
            # We flip with respect to the prior qubit state
            current_state = 1 - current_state
        # Store the state of this qubit
        qubits_state.append(current_state)

    # If, as we assumed, q0 has not flipped, then our qubit_state tells us
    # which qubtis have flipped.
    # But if q1 has flipped, then our qubit_state is the opposite
    # of the actual flip-state of each qubit.

    # If we have more 1s than 0s, that is a very unlikely scenario
    # (2 bits flipped)so we can infer that q1 had actually flipped
    # (and our qubits_state is the opposite of the actual flip-state).
    num_ones = sum(qubits_state)
    num_zeros = len(qubits_state) - num_ones

    # We now need to make our flip-prediction for the observable.
    # Stim's circuit will contain the observable measurement in the last qubit
    # of the block, because it uses the last measurement.
    if num_ones > num_zeros:
        # If we have more 1s than 0s, we assume that our flip-states are inverted,
        # so we take the opposite of the last qubit's flip-state
        # as our prediction for the observable.
        local_prediction = 1 - qubits_state[-1]
    else:
        # If we have more 0s than 1s, we assume our flip-states are correct.
        # So the flip-state of the last qubit is the right flip-state
        # for the observable.
        local_prediction = qubits_state[-1]

    shot_predictions.append(local_prediction)
```

Listing 5.2. *Simulation 2: Implementation of a simple Majority Vote decoder for a repetition code*

As seen in Figure 5.1, for simple repetition codes under basic noise models, both decoders perform similarly, effectively reducing physical errors. In the following section we will stress-test our decoders to see how their efficiency decays with different noise models.

5.1.3 Introducing circuit-level noise

While the code-capacity model is useful for validating decoders, it does not reflect physical reality. In a real quantum computer, the gates used to entangle the qubits and the measurements used to extract the syndrome are themselves noisy.

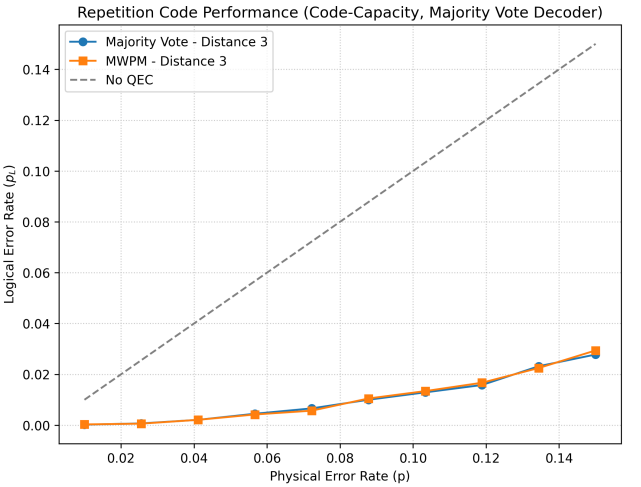


Figure 5.1. *Simulation 3: Comparison of the results of simulations 1 and 2. Displays the logical error for a basic repetition code using Majority Vote and MWPM decoders. The x=y dotted line helps comparing the QEC protocol to not implementing any QEC. If the logical error rate is above the physical error rate, the QEC protocol is adding errors*

To illustrate this, we expand our simulation to include **circuit-level noise**. As seen in Chapter 3, in this model, we inject depolarizing noise into single-qubit and two-qubit gates, and we add a probability of measurement readout errors. For the purpose of this test, we will use a very simple noise model, where the three types of noise added to the circuit have the same probability (ratio 1:1:1). This is easily implemented with `stim` in the circuit creation, as seen in Listing 5.3.

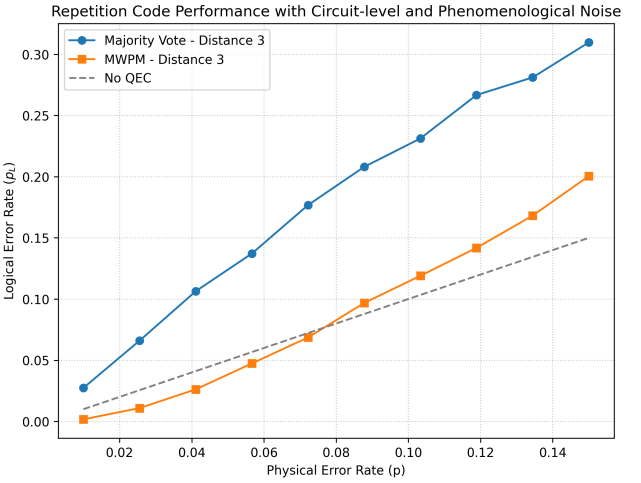


Figure 5.2. *Simulation 4: Impact of circuit-level and phenomenological noise on the repetition code performance. Introducing more complex noise significantly degrades the logical error suppression compared to the idealized code-capacity baseline. With this additional noise, MWPM shows its advantage over the Majority Vote because of its capacity to consider the probabilities of each error to select the most probable set of errors that could have caused the observed syndrome.*

Source: `src/chapter_5/s4_majority_vote_noise.py`

```
circuit = stim.Circuit.generated(
    "repetition_code:memory",
    distance=distance,
    rounds=1,
    before_round_data_depolarization=p,
    after_clifford_depolarization=p,
    before_measure_flip_probability=p
)
```

Listing 5.3. *Simulation 4: Adding circuit-level noise.*

As shown in Figure 5.3, realistic noise drastically reduces the effectiveness of the code, but it also shows the importance of the decoder. Handling circuit-level noise requires complex spatial-temporal decoding graphs, which makes MWPM a better solution than a basic Majority Vote decoder. The figure also shows how increasing the distance improves the results of the MWPM decoder, but it degrades the result of the Majority Vote decoder. This means that, for this specific noise model and the MWPM decoder, the system is operating under the threshold, so the error correcting capacity of the protocol can be increased by increasing the distance. However, with our Majority Vote decoder, the system operates above the threshold.

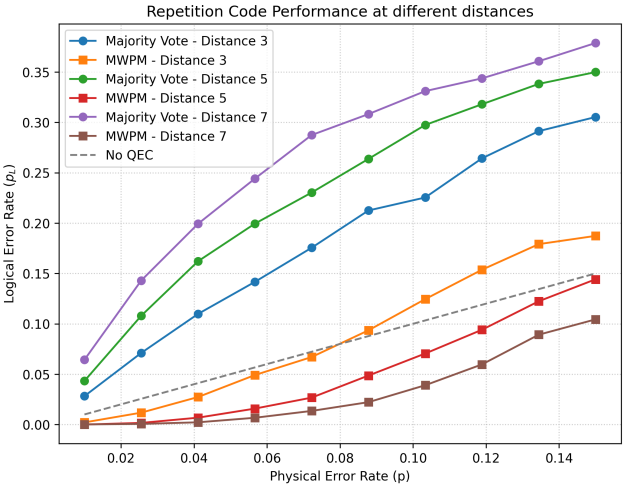


Figure 5.3. *Simulation 5: Under a complex noise model (code-capacity, circuit-level and phenomenological) increasing the distance of the code has the opposite effect for each of the decoders.*

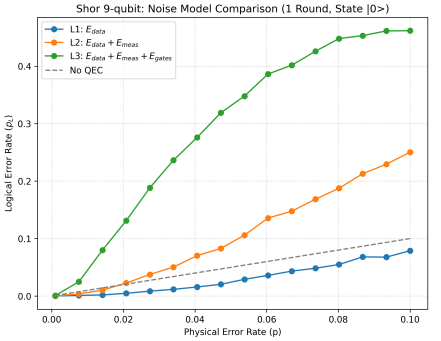
5.1.4 Implementing the 9-qubit Shor code

Because the final measurement of the data qubits can also be noisy, we must add one last set of detectors at the end of the circuit. These detectors, known as boundary detectors, compare the parity of the final data measurements against the last syndrome extracted via ancillas, effectively catching readout errors in the final layer.

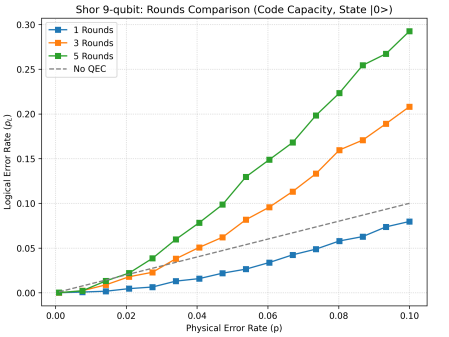
In our previous simulations, we relied on `stim`'s built-in circuit generation library. To better illustrate how a quantum error correction circuit is explicitly constructed from scratch, we now implement the 9-qubit Shor code ($[[9, 1, 3]]$). Building a custom circuit in `stim` follows a strict pipeline.

First, the circuit is declared and initialized, followed by the application of the necessary gates to encode the logical state and extract the stabilizer operators. A crucial step in custom `stim` circuits is explicitly tracking the outcomes: stabilizer measurements must be labeled as `DETECTOR`s (Listing 5.7), while the final measurements of the data qubits are

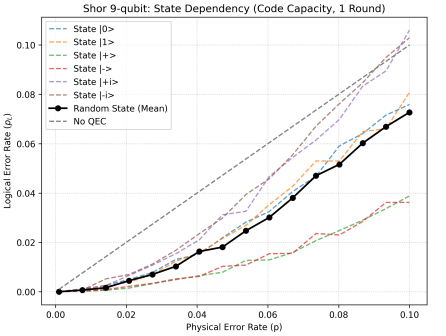
targeted by `OBSERVABLE_INCLUDE` instructions to track the logical state parity (Listing 5.8). This strict labeling allows the simulator to decouple the syndrome extraction—which provides the inputs for the decoder—from the logical observable tracking, which is ultimately compared against the expected baseline to determine if a logical failure occurred.



(a) *Noise Models*



(b) *Rounds*



(c) *State Dependency*

Figure 5.4. ***Simulation 6****: Performance analysis of the Shor 9-qubit code using the MWPM decoder.* ***(a) Noise models****: Comparison between Code-capacity, Phenomenological, and Circuit-level noise.* ***(b) Rounds****: Accumulation of errors over syndrome measurement rounds.* ***(c) State Dependency****: Performance for different initial states. When measuring in the Z-basis ($|0\rangle$ or $|1\rangle$), the logical outcome is insensitive to logical phase-flips (physical X errors) but vulnerable to logical bit-flips (physical Z errors). Conversely, measurements in the X-basis ($|+\rangle$ or $|-\rangle$) are vulnerable to logical phase-flips but insensitive to logical bit-flips. Due to the code's concatenated structure, there are fewer fatal combinations for physical X errors than for physical Z errors, resulting in higher logical protection for the $|\pm\rangle$ states. Finally, measurements in the Y-basis ($|\pm i\rangle$) are vulnerable to fatal combinations of both physical X and physical Z errors, effectively accumulating the vulnerabilities of both previous cases and resulting in the highest logical failure rate.*

Finally, to evaluate the code under realistic conditions, noise must be integrated into the circuit. Unlike the automated generator, building a circuit from scratch requires manually injecting specific noise channels-such as gate depolarization, measurement flips, or idle decoherence-at the exact coordinates where they occur (Listings 5.4, 5.5, and 5.6).

To illustrate the impact of different noise models, our initial analysis compares the performance of the Shor code under code-capacity, phenomenological, and circuit-level noise. The results demonstrate how it is highly vulnerable to error from noisy gates and faulty measurements, degrading its error-correcting capabilities.

We also analyze the accumulation of errors across multiple syndrome extraction rounds. Our results show that the logical error rate scales rapidly with each round, making the code unsuitable for long computations. While the standard solution to this would be increasing the distance, the Shor code does not scale well: increasing the distance increases the number of necessary qubits exponentially and also increases the number of qubits that must be measured by a single detector. Furthermore, the code is not robust against noisy gates or measurements. Topological codes, which we simulate in the following sections, offer solutions to these problems through improved scaling and the capacity to handle time-dependent noise.

To evaluate how the code performs against different error types, we could have explicitly biased the physical noise channels. However, by injecting uniform depolarizing noise and altering the initial states instead, we preserve a standardized noise model while probing specific vulnerabilities.

Finally, by preparing the system in different cardinal states before measurement, we expose the logical observable to different types of uncorrected physical errors. This methodological choice empirically reveals how the code performs asymmetrically due to the inherent characteristics of its concatenated structure.

Source: `src/chapter_5/s6_shor_9_code.py`

```python
def append_noisy_gate(circuit: stim.Circuit, gate: str, targets: list, noise: float) -> None:
    """Appends a gate and adds depolarizing noise if noise > 0."""
    circuit.append(gate, targets)
    if noise > 0:
        if gate in ["CX", "CY", "CZ", "SWAP"]:
            circuit.append("DEPOLARIZE2", targets, noise)
        else:
            circuit.append("DEPOLARIZE1", targets, noise)
```

Listing 5.4. *Simulation 6: Injecting circuit-level noise by appending error channels to quantum gates.*

Source: `src/chapter_5/s6_shor_9_code.py`

```python
def append_noisy_measurement(circuit: stim.Circuit, targets: list, noise: float) -> None:
    """Appends a measurement with pre-measurement bit-flip noise."""
    if noise > 0:
        circuit.append("X_ERROR", targets, noise)
    circuit.append("M", targets)
```

Listing 5.5. *Simulation 6: Simulating readout errors by assigning flip probabilities to measurement operations.*

Source: `src/chapter_5/s6_shor_9_code.py`

```python
# Syndrome measurement rounds
for r in range(rounds):
    if before_round_data_depolarization > 0:
        circuit.append("DEPOLARIZE1", range(9), before_round_data_depolarization)
```

Listing 5.6. *Simulation 6: Applying data qubit decoherence during idle cycles. Each round represents a time step where the logical state must be preserved, followed by detector evaluations to track newly occurring errors.*

Source: `src/chapter_5/s6_shor_9_code.py`

```python
append_noisy_measurement(circuit, range(9, 15), before_measure_flip_probability)
for i in range(6):
    if r == 0: circuit.append("DETECTOR", [stim.target_rec(-6 + i)])
    else: circuit.append("DETECTOR", [stim.target_rec(-6 + i), stim.target_rec(-6 + i - 8)])
```

Listing 5.7. *Simulation 6: Declaring circuit detectors. Following a stabilizer measurement, a DETECTOR is defined by comparing the current outcome with the corresponding record from the previous round to identify parity changes.*

Source: `src/chapter_5/s6_shor_9_code.py`

```
append_noisy_measurement(circuit, range(9), before_measure_flip_probability)

# Boundary detectors and observables
if basis == "X":
    for i, targets in enumerate([(0,1), (1,2), (3,4), (4,5), (6,7), (7,8)]):
        circuit.append("DETECTOR", [stim.target_rec(-17 + i), stim.target_rec(-9 + targets
      [0]), stim.target_rec(-9 + targets[1])])
    # Logical X Distinguishes + and - (Measure Z0*Z3*Z6)
    circuit.append("OBSERVABLE_INCLUDE", [stim.target_rec(-9), stim.target_rec(-6), stim.
      target_rec(-3)], 0)
elif basis == "Z":
    circuit.append("DETECTOR", [stim.target_rec(-11)] + [stim.target_rec(-i) for i in range
      (4, 10)])
    circuit.append("DETECTOR", [stim.target_rec(-10)] + [stim.target_rec(-i) for i in range
      (1, 7)])
    # Logical Z Distinguishes 0 and 1 (Measure transversal X parity)
    circuit.append("OBSERVABLE_INCLUDE", [stim.target_rec(-i) for i in range(1, 10)], 0)
else:
    # For basis Y, we just measure transversal parity
    circuit.append("OBSERVABLE_INCLUDE", [stim.target_rec(-i) for i in range(1, 10)], 0)
```

Listing 5.8. *Simulation 6: Final destructive measurement of the data qubits, including the explicit declaration of boundary detectors and the logical observable target.*

5.1.5 Visualizing the Threshold Theorem

To conclude our exploration of baseline models, we return to the repetition code to visually demonstrate one of the most fundamental concepts in quantum error correction: the Threshold Theorem. As discussed theoretically in Chapter 1, this theorem guarantees that if the physical error rate is below a certain threshold p_{th}, we can arbitrarily reduce the logical error rate by increasing the distance of the code.

To observe this transition clearly in a simulation, two critical adjustments are required. First, we must adopt an appropriate noise model. Under pure code-capacity noise, the 1D repetition code lacks a standard visible threshold because the decoder can adapt to extreme error rates with perfect syndromes. We opt for a noise model that is fundamentally code-capacity, but we inject small rates of circuit-level noise (specifically, small probabilities of measurement and gate errors, at a 1:0.05:0.05 ratio). This forces the decoder to handle imperfect syndromes, allowing us to see exactly how the code behaves when its limits are tested.

The noise model in this simulation has been specifically selected to cause a visible threshold while having a logical error rate that is lower than the physical error rate. Circuit-level noise is required for the code to have a visible threshold. Such noise needs to be at a very low rate because a simple repetition code does not perform well and quickly saturates at the maximum logical noise.

Second, we must ensure that the number of syndrome extraction rounds grows proportionally with the code distance (setting `rounds = d`). Fault tolerance must be achieved in both space (number of qubits) and time (measurement rounds). If we were to increase the number of qubits but only measure the syndrome once, the code would be heavily protected spatially but extremely fragile temporally. By increasing the rounds along with the distance, we force the larger codes to survive the noise for a longer period. While this might intuitively seem unfair—giving larger codes more opportunities to fail—the power of the Threshold Theorem is that, below the threshold, the error suppression is so strong that the larger code still achieves a lower total error rate despite enduring more rounds of noise. Figure 5.5 demonstrates this effect perfectly.

5.2 Beyond stim: Experimental setup for implementation and simulation

As demonstrated in the previous sections, `stim` is an exceptionally powerful tool for defining and sampling quantum circuits. However, when transitioning from baseline models to complex, scalable topological codes like the Surface Code or the Color Code, relying solely on `stim` becomes insufficient.

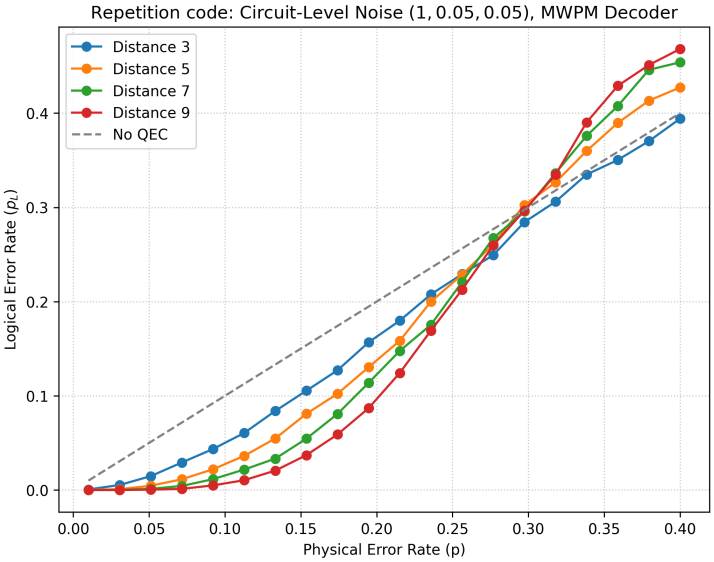


Figure 5.5. *Simulation 7: Visualizing the Threshold Theorem using a repetition code under a predominantly code-capacity noise model with small rates of circuit-level noise (ratio 1:0.05:0.05). For physical error rates below the threshold (the point where the curves intersect), increasing the distance of the code strictly improves its performance, driving the logical error rate down. Conversely, for physical error rates above the threshold, the noise overwhelms the system, and adding more qubits only introduces more errors, degrading the overall performance.*

To efficiently perform large-scale research, we must introduce two additional abstraction layers on top of `stim`: one for orchestrating massive parallel simulations, and another for procedurally generating highly complex, geometrically constrained circuits.

5.2.1 Sampling technologies: `sinter`

When evaluating QEC protocols below their threshold, the logical error rate decays exponentially. To observe statistically significant failure rates at extremely low physical error probabilities (e.g., 10^{-6}), billions of circuit shots must be sampled. Executing these operations sequentially in standard Python loops is computationally unfeasible.

To solve this, we use **`sinter`**, a specialized tool designed to orchestrate and parallelize `stim` simulations. Sinter's primary advantages are:

- **Automatic parallelization**: It seamlessly distributes the sampling tasks across all available CPU cores.
- **Dynamic stopping criteria**: Instead of executing a fixed number of shots, `sinter` allows defining a maximum number of logical errors (e.g., `max_errors=500`). The simulation will automatically halt for a given data point once sufficient statistical confidence is reached, saving immense computational time.
- **Data management**: It automatically aggregates the results into structured CSV files, mitigating the risk of data loss during long-running simulations.

Listing 5.9 illustrates how to configure and execute a massively parallel simulation using `sinter`.

Source: `src/chapter_5/s8_beyond_stim.py`

```python
# Define the tasks we want Sinter to simulate
tasks = []
if distances is None:
    distances = [3, 5, 7]
if physical_errors is None:
    physical_errors = np.linspace(0.01, 0.4, 20)

for d in distances:
    for p in physical_errors:
        circuit = build_rep_code_cirq(distance=d, p=p)
        tasks.append(
            sinter.Task(
                circuit=circuit,
                json_metadata={'d': d, 'p': p}
            )
        )

# Collect statistics using Sinter in parallel
print("Starting Sinter parallel sampling...")
stats = sinter.collect(
    num_workers=num_workers,
    tasks=tasks,
    decoders=['pymatching'],
    max_shots=max_shots,
    max_errors=max_errors,
    print_progress=True
)
```

Listing 5.9. *Simulation 8: Executing a large-scale parallel simulation using sinter. The tool samples up to a million shots, but halts early if 500 errors are found. Sinter automatically parallelizes using multiple workers.*

5.2.2 High level circuit design for quantum error correction: `cirq` and `stimcirq`

While `stim` includes built-in generators for Surface Codes, it lacks native support to generate Color Codes. Constructing a generic topological code directly in `stim` requires manually managing numerical qubit indices and their intricate spatial interactions, a process that is highly error-prone.

To overcome this limitation, we utilize **`cirq`** [29], Google's high-level framework for quantum circuits. `cirq` provides critical advantages for QEC design:

- **Geometric layout**: Qubits can be defined using planar coordinates (e.g., `cirq.GridQubit(x, y)`). This will allow us to map the hexagonal or triangular faces of a Color Code directly into the circuit's architecture.
- **High-level abstractions**: Because of the mentioned geometric layout, operations can be designed geometrically, making syndrome extraction routines more intuitive.

However, mapping a high-level `cirq` circuit into a fault-tolerant simulation requires tracking metadata across the temporal and space dimensions. As shown in Listing 5.10, we must use the **`stimcirq`** integration library to manually annotate the circuit. This involves defining unique string keys for every measurement, strictly placing annotations in the right circuit tick (`cirq.Moment`), and explicitly declaring `stimcirq.DetAnnotation` to compare parity checks across rounds.

Once the circuit is constructed and annotated, it compiles into an ultra-fast `stim` circuit, granting us both the expressive power of `cirq` and the raw performance of `stim`.

Source: `src/chapter_5/s8_beyond_stim.py`

```python
qubits = cirq.LineQubit.range(2 * distance - 1)
circuit = cirq.Circuit()
rounds = distance

for r in range(rounds):
    # Add noise to data qubits
    # the NEW_THEN_INLINE strategy is used to prevent the noise annotation to be pushed to the earliest
    # possible step, which would result in an incorrect simulation.
    circuit.append([cirq.depolarize(p).on(q) for q in qubits[0::2]], strategy=cirq.InsertStrategy.NEW_THEN_INLINE)

    # CNOTs to compute parity -- First qubit is a data qubit. Then we alternate ancillas and qubits. Each ancilla
    # is connected to two data qubits, one on each side.
    # Odd qubits are ancillas, even qubits are data qubits.
    circuit.append([cirq.CNOT(qubits[i-1], qubits[i]) for i in range(1, 2*distance-1, 2)], strategy=cirq.InsertStrategy.NEW_THEN_INLINE)
    circuit.append([cirq.CNOT(qubits[i+1], qubits[i]) for i in range(1, 2*distance-1, 2)], strategy=cirq.InsertStrategy.NEW_THEN_INLINE)

    # Measure ancillas (with explicit string keys for stimcirq tracking)
    circuit.append([cirq.measure(qubits[i], key=f'm_{r}_{i}') for i in range(1, 2*distance-1, 2)], strategy=cirq.InsertStrategy.NEW_THEN_INLINE)

    # Define Detectors using stimcirq.DetAnnotation
    det_ops = []
    for i in range(1, 2*distance-1, 2):
        key = f'm_{r}_{i}'
        if r == 0:
            det_ops.append(stimcirq.DetAnnotation(parity_keys=[key], coordinate_metadata=(i, r)))
        else:
            prev_key = f'm_{r-1}_{i}'
            det_ops.append(stimcirq.DetAnnotation(parity_keys=[key, prev_key], coordinate_metadata=(i, r)))
    circuit.append(det_ops, strategy=cirq.InsertStrategy.NEW_THEN_INLINE)

    # Reset ancillas for the next round
    circuit.append([cirq.reset(qubits[i]) for i in range(1, 2*distance-1, 2)], strategy=cirq.InsertStrategy.NEW_THEN_INLINE)

# Final Data measurement (of data qubits)
circuit.append([cirq.measure(qubits[i], key=f'd_{i}') for i in range(0, 2*distance-1, 2)], strategy=cirq.InsertStrategy.NEW_THEN_INLINE)

# Final Detectors (comparing data parity to last ancilla measurement)
# For each ancilla we add the ancilla measurement from prior round and the measurement of the two data qubits that it is connected to.
final_det_ops = []
for i in range(1, 2*distance-1, 2):
    ancilla_key = f'm_{rounds-1}_{i}'
    data_left = f'd_{i-1}'
    data_right = f'd_{i+1}'
    final_det_ops.append(stimcirq.DetAnnotation(
        parity_keys=[ancilla_key, data_left, data_right],
        coordinate_metadata=(i, rounds)
    ))
circuit.append(final_det_ops, strategy=cirq.InsertStrategy.NEW_THEN_INLINE)

# Define Logical Observable
# The observable is simply the first data qubit
circuit.append(stimcirq.CumulativeObservableAnnotation(
    parity_keys=[f'd_0'],
    observable_index=0
), strategy=cirq.InsertStrategy.NEW_THEN_INLINE)
```

Listing 5.10. *Simulation 8: Using cirq to procedurally generate a fully functional Repetition Code. The circuit is manually annotated with noise models, stimcirq.DetAnnotation, and stimcirq.CumulativeObservableAnnotation to define the spatial-temporal parity checks required by decoders, ensuring the translation to stim is exact.*

5.3 Automated noise injection: Extracting error models from Qiskit Fake Providers

In the preceding sections, we simulated circuit-level noise by manually injecting depolarizing channels and measurement flips at specific coordinates in the `stim` circuit. While this illustrates the mechanics of noise, it does not scale when we try to construct complex QEC codes.

Fortunately, `cirq` provides a robust mechanism to automate this process: the `cirq.NoiseModel` allows us to apply a set of noise rules to every operation in a circuit. To demonstrate its power, we will extract physical error rates from a real quantum processor using IBM's Qiskit `FakeProviders`, and automatically inject them into our simulations.

5.3.1 Extracting physical parameters

Using Qiskit's backend properties, we can extract the specific error rates of a physical chip. For our simulation, we compute the average single-qubit gate error (p_{1q}), two-qubit gate error (p_{2q}), and measurement readout error (p_{meas}).

Listing 5.11 demonstrates how these average parameters are programmatically extracted from a Qiskit backend.

In a strictly exhaustive circuit-level noise model, one should also include idle noise. Because two-qubit gate errors (p_{2q}) are typically orders of magnitude higher than idle decoherence during the gate's duration, we approximate the system by only applying noise directly to active gates and measurements.

Source: `src/chapter_5/s9_qiskit_noise_model.py`

```python
def get_average_error_rates(backend):
    """Extracts average error rates from a Qiskit backend."""
    target_1q = []
    target_2q = []
    target_meas = []

    for q in range(backend.num_qubits):
        meas_props = backend.target.get("measure", None)
        if meas_props is not None and (q,) in meas_props:
            target_meas.append(meas_props[(q,)].error)

        sx_props = backend.target.get("sx", None)
        if sx_props is not None and (q,) in sx_props:
            target_1q.append(sx_props[(q,)].error)

    cx_props = backend.target.get("cx", None)
    if cx_props is not None:
        for edge in cx_props:
            target_2q.append(cx_props[edge].error)

    p_1q = np.mean(target_1q) if target_1q else 0.001
    p_2q = np.mean(target_2q) if target_2q else 0.01
    p_meas = np.mean(target_meas) if target_meas else 0.02

    return p_1q, p_2q, p_meas
```

Listing 5.11. *Simulation 9: Extracting average physical error rates from a Qiskit FakeProvider.*

5.3.2 Automating noise injection with `cirq.NoiseModel`

Once the physical parameters are extracted, we must map them into our `cirq` workflow. By inheriting from `cirq.NoiseModel`, we can override the `noisy_operation` method to define specific rules for each gate type.

As shown in Listing 5.12, we apply bit-flips to measurements and depolarizing channels to gates.

Source: `src/chapter_5/s9_qiskit_noise_model.py`

```python
class IBMNoiseModel(cirq.NoiseModel):
    """Custom Cirq noise model applying extracted error rates."""
    def __init__(self, p_1q, p_2q, p_meas, scale=1.0):
        self.p_1q = min(p_1q * scale, 1.0)
        self.p_2q = min(p_2q * scale, 1.0)
        self.p_meas = min(p_meas * scale, 1.0)

    def noisy_operation(self, operation: cirq.Operation):
        if isinstance(operation, (stimcirq.DetAnnotation, stimcirq.
     CumulativeObservableAnnotation)):
            yield operation
            return

        if cirq.is_measurement(operation):
            yield cirq.bit_flip(p=self.p_meas).on_each(*operation.qubits)
            yield operation
        elif len(operation.qubits) == 1:
            yield operation
            yield cirq.depolarize(p=self.p_1q).on_each(*operation.qubits)
        elif len(operation.qubits) == 2:
            yield operation
            yield cirq.depolarize(p=self.p_2q).on_each(*operation.qubits)
        else:
            yield operation
```

Listing 5.12. *Simulation 9: Implementing a custom cirq.NoiseModel that applies the extracted error rates. The scaled error rates are bounded to avoid exceeding 1.0.*

To run our simulations, we often want to plot physical noise in the X axis. Since our extracted Qiskit noise provides a single fixed point, we introduce a `scale` factor. We multiply the base error rates by this factor to artificially worsen or improve the hardware's performance, allowing us to trace the error curve while maintaining the realistic relative ratios between single-qubit, two-qubit, and measurement errors.

When plotting these results, we must select a single metric for the X-axis to represent the physical error rate. We use the two-qubit gate error rate (p_{2q}) as the reference.

Listing 5.13 shows how we build a perfectly clean circuit, use the `with_noise` method to automatically inject noise in the circuit based on our IBM model.

Source: `src/chapter_5/s9_qiskit_noise_model.py`

```python
noise_model = IBMNoiseModel(base_1q, base_2q, base_meas, scale=scale)
noisy_circuit = clean_circuit.with_noise(noise_model)
stim_circuit = stimcirq.cirq_circuit_to_stim_circuit(noisy_circuit)
```

Listing 5.13. *Simulation 9: Applying the noise model to a clean circuit and converting it to stim for large-scale sampling.*

CHAPTER 6

Simulating 2D triangular color codes with a 4.8.8 lattice geometry and a code-capacity noise model

6.1 Syndrome Graph Construction and Decoding

6.1.1 The Detector Error Model (DEM)

As seen in Chapter 4, many modern decoders rely on mapping the decoding problem to a classical graph-theory problem. A key element of this framework is the **Detector Error Model**, which represents the error vulnerabilities with a specific quantum circuit. The DEM is constructed by `stim` before the simulation begins and passed to the decoder. It is constructed as an hypergraph:

Unlike standard graphs, in an hypergraph a hyper-edge can connect any number of nodes. That allows the edges in the DEM to connect one single physical errors to any number of detectors (e.g., 1, 2, or 3).

- **Nodes (Detectors):** Each node represents a deterministic parity check (a *detector*), which evaluates to zero under noiseless conditions (e.g., the comparison of a stabilizer measurement between cycle t and $t-1$).
- **Hyperedges (Errors):** A hyperedge connects the specific set of detectors flipped by a single, independent physical error.
- **Weights:** Each hyperedge is assigned a weight calculated as $w = \ln((1-p)/p)$, where p is the physical probability of that specific fault occurring.
- **Observable Payload:** Each hyperedge carries a boolean payload indicating whether the physical error causes a flip in the logical state (which means this specific physical error operation anti-commutes with the logical observable).

`stim` automates the generation of the DEM through fast symbolic simulation:

1. **Independent physical errors (nodes)**: `stim` identifies every possible error in the circuit, which is defined by the error channels that have been injected in the circuit definition. This includes code-capacity, gate and measurement errors.
2. **Error simulation:** For each independent error identified, `stim` evaluates all the detectors and logical observables in the circuit. Instead of running a computationally expensive full simulation of the circuit, `stim` mathematically propagates

each error forward through the circuit. Because code-correction circuits contain exclusively Clifford gates, determining how the error propagates is efficient.

3. **Logical error characterization (hyperedges):** `stim` observes exactly which detectors and logical observables are inverted by the propagated error. It then adds a hyperedge to the DEM connecting all the flipped detectors, alongside its probability and observable payload.

6.1.2 Decoding the DEM

In the simulation process, for each experiment shot, **stim** generates errors with the probability established by the error model and produces the syndrome (the list of detectors that have been activated) and the observable that is measured at the end of the execution.

The DEM serves as the static blueprint for the decoder. Given a certain syndrome, the decoder must determine the physical error (or combination of multiple physical errors, which we call the error chain) that caused such syndrome. Because different error chains can cause a given syndrome, the problem consist on determining the one that has the highest probability of occurring.

Once the most probable error chain is identified, the decoder computes the collective parity of their observable payloads (i.e. which of the physical errors in the chain cause a flip in the observable). If the final parity is 1, it predicts a logical flip. This prediction is compared against the actual logical observable measured by Stim to determine the success or failure of the decoding cycle.

6.2 Implementing a 2D color code

To systematically implement the 2D Color Code and allow for future scalability, we have adopted a modular software architecture. The problem has been divided into two strictly independent layers: the topological construction of the grid (*Lattice Construction*) and the quantum logic representation (*Circuit Generation*). This separation of concerns ensures that the circuit generator is completely agnostic to the underlying geometry; it simply receives a set of abstract shapes and coordinates, meaning new lattices (such as the 6.6.6 hexagonal grid) can be added in the future without altering the core quantum logic.

Furthermore, this approach simplifies visualizing the lattice geometry and confirming correctness before having to dive into the circuit implementation.

6.2.1 Lattice Construction

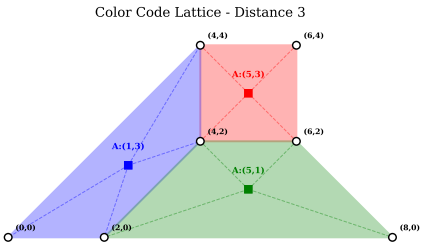
The abstract class `Lattice` serves as the foundation. It defines the generic behavior of a 2D topological grid, treating it simply as a collection of geometric shapes. Each shape inherits from a base `Shape` class and defines its own data qubits, its ancilla (detector) qubit, and its color.

The specific implementation for this work is the `Lattice488` class, which procedurally generates the 4.8.8 semi-regular lattice (consisting of squares and octagons). The algorithm builds the lattice level by level:

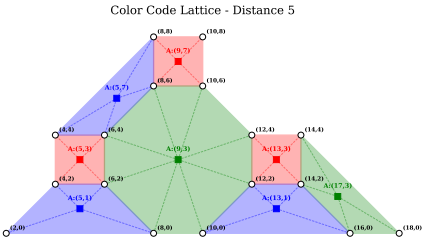
1. It first places the `SquareShape` instances in a pyramid-like structure.
2. It then fills the gaps between these squares with `OctogonShape` instances.
3. Finally, to ensure all boundary data qubits are protected by valid stabilizers, it caps the left, right, and bottom boundaries with `TrapezoidShape` instances.

By dynamically calculating coordinates and alternating colors based on the distance d, the lattice scales automatically to any arbitrary size. Figure 6.1 shows the automated geometric evolution of the 4.8.8 lattice for distances 3, 5, 7 and 9.

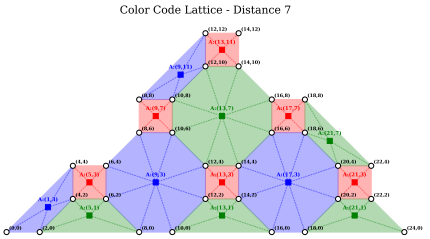
To facilitate converting the lattice in a circuit, and inspired by the mapping of a 4.8.8. geometry to a square grid proposed by Fowler [19] and discussed in Chapter 2, we assign grid coordinates to each data and ancilla qubit in the grid. The geometry and the specific grid mapping results in the detectors having odd x and y coordinates, and data qubits having even x and y coordinates.



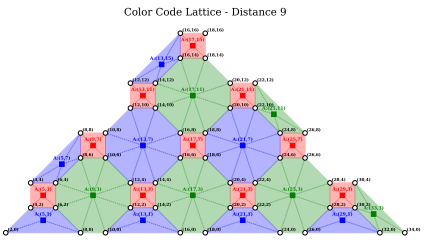
(a) $d = 3$



(b) $d = 5$



(c) $d = 7$



(d) $d = 7$

Figure 6.1. *Evolution of the 4.8.8 Color Code lattice for distances 3, 5, 7 and 9. The solid squares represent the ancilla qubits (syndrome detectors) positioned at the center of each face. The white circles with black outlines represent the physical data qubits. The labels indicate their specific (x, y) coordinates in the simulated grid. High resolution versions of these images are available in the github repository (see Page vii)*

6.2.2 Circuit Generation

With the geometry resolved, the `ColorCode` class constructs the quantum circuit. It receives a `Lattice` object and extracts all unique coordinates, translating them into `cirq.GridQubit` objects.

Unlike a manual layout, this class dynamically maps the geometric shapes to stabilizer measurement routines. For each syndrome extraction round, it prepares the ancillas, applies Hadamard gates where necessary, and executes CNOT gates between each ancilla and its surrounding data qubits. Finally, it performs a measurement of the data qubits to extract the logical observable.

Under a code-capacity error model, the order in which CNOT gates are performed is not relevant. However, under more realistic error models, physical errors can occur during syndrome extraction and propagate through subsequent CNOT gates, causing correlated multi-qubit failures known as hook errors. This phenomenon requires a carefully planned CNOT schedule.

Detectors are defined after each syndrome extraction. To reduce the number of qubits, ancilla qubits are reused for the X and Z stabilizer measurement, which are sequential. It's worth noting that in the first round X detectors are not added. This is because, with the initial state of the qubits set as $|0\rangle$, measurements in the X axis are not deterministic, so the measured syndrome would be random. After the first round of syndrome measurements, qubit state collapses to a defined value, which can then be used in subsequent rounds to detect phase-flip errors.

Similarly, the observable is defined at the end of the circuit. While in the 2D color codes observables are generally defined as a string-nets connecting the boundaries

of the lattice (and with minimal weight), an observable defined by all the data qubits is topologically equivalent. This allows us to implement a very simple definition of the observable.

Figure 6.2 illustrates the resulting quantum circuit for a distance 3 code with a single measurement round.

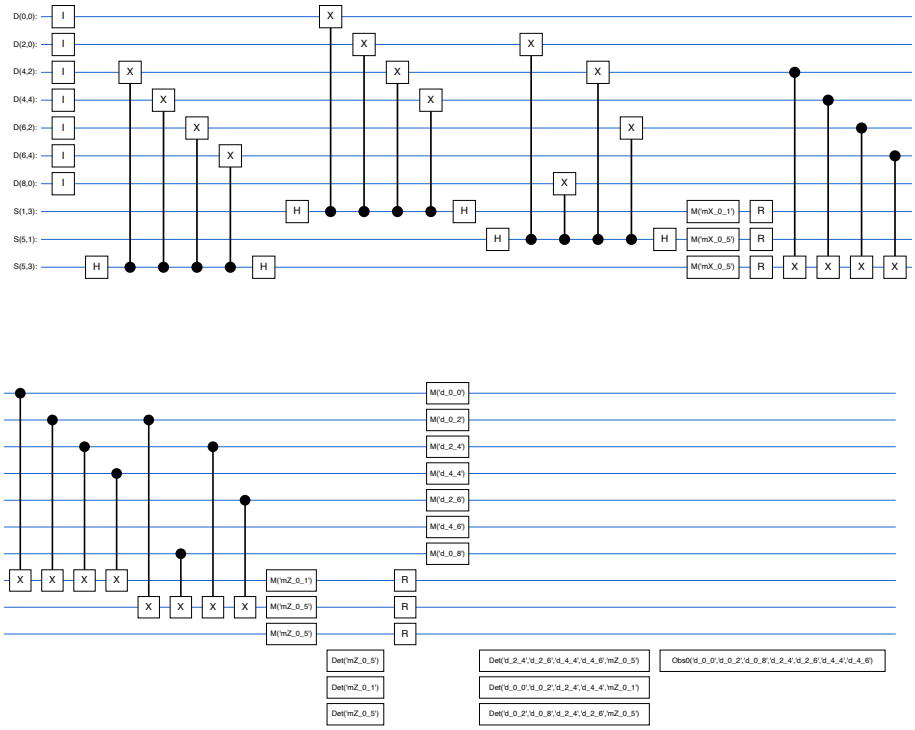


Figure 6.2. *Complete quantum circuit for the 4.8.8 Color Code ($d = 3$, 1 round). The data qubits have been labeled as $D(x, y)$ and the ancilla qubits for the stabilizers as $S(x, y)$, so they are grouped to ease the visualization. Due to its significant width, the image has been dynamically split into two halves for readability. High resolution versions of these images are available in the github repository (see Page vii)*

6.3 BP+OSD

6.4 Projection decoder + MWPM/Union-find

6.5 Restriction decoder + MWPM

6.6 Möbius Decoder + MWPM

6.7 Concatenated MWPM decoder

6.8 Correlated matching decoder

CHAPTER 7

Comparative analysis and discussion

CHAPTER 8

Evaluating potential improvements to projection decoding for 2D color codes

Nota: para metodología de la investigación hay que buscar una revista en la que tuviese sentido publicar tu TFM.

Esta revista que está bastante especializada y aceptan artículos tipo tutorial, en lo que podría encajar: https://tqe.ieee.org/special-section-on-quantum-engineering-education/

Otras opciones serían: PRX Quantum (Sección “Tutorials”)

Contents

List of Figures

Bibliography

[1] Michael A Nielsen and Isaac L Chuang. *Quantum Computation and Quantum Information*. Cambridge University Press, 2010.

[2] Richard P Feynman. “Simulating physics with computers”. In: *International Journal of Theoretical Physics* 21.6/7 (1982).

[3] Peter W Shor. “Algorithms for quantum computation: discrete logarithms and factoring”. In: *Proceedings 35th Annual Symposium on Foundations of Computer Science*. IEEE. 1994, pp. 124–134.

[4] Peter W. Shor. “Scheme for reducing decoherence in quantum computer memory”. In: *Physical Review A* 52.4 (1995), R2493–R2496. DOI: 10.1103/PhysRevA.52.R2493. URL: https://link.aps.org/doi/10.1103/PhysRevA.52.R2493.

[5] Andrew M. Steane. “Error correcting codes in quantum theory”. In: *Physical Review Letters* 77.5 (1996), pp. 793–797. DOI: 10.1103/PhysRevLett.77.793. URL: https://link.aps.org/doi/10.1103/PhysRevLett.77.793.

[6] A Yu Kitaev. “Fault-tolerant quantum computation by anyons”. In: *Annals of Physics* 303.1 (2003), pp. 2–30.

[7] H. Bombin and M. A. Martin-Delgado. “Topological Quantum Distillation”. In: *Physical Review Letters* 97.18 (Oct. 2006). ISSN: 1079-7114. DOI: 10.1103/physrevlett.97.180501. URL: http://dx.doi.org/10.1103/PhysRevLett.97.180501.

[8] Craig Gidney. “Stim: a fast stabilizer circuit simulator”. In: *Quantum* 5 (2021), p. 497. DOI: 10.22331/q-2021-07-06-497.

[9] William K. Wootters and Wojciech H. Zurek. “A single quantum cannot be cloned”. In: *Nature* 299.5886 (1982), pp. 802–803. DOI: 10.1038/299802a0.

[10] Emanuel Knill and Raymond Laflamme. “Theory of quantum error-correcting codes”. In: *Physical Review A* 55.2 (1997), p. 900. DOI: 10.1103/PhysRevA.55.900. URL: https://doi.org/10.1103/PhysRevA.55.900.

[11] Daniel Gottesman. “Stabilizer codes and quantum error correction”. arXiv preprint quant-ph/9705052. PhD thesis. California Institute of Technology, 1997. URL: https://arxiv.org/abs/quant-ph/9705052.

[12] Dorit Aharonov and Michael Ben-Or. “Fault-tolerant quantum computation with constant error”. In: *Proceedings of the twenty-ninth annual ACM symposium on Theory of computing*. 1997, pp. 176–188. DOI: 10.1145/258533.258579.

[13] Bryan Eastin and Emanuel Knill. “Restrictions on transversal encoded quantum gate sets”. In: *Physical Review Letters* 102.11 (2009), p. 110502.

[14] Sergey Bravyi and Alexei Kitaev. “Universal quantum computation with ideal Clifford gates and noisy ancillas”. In: *Physical Review A* 71.2 (2005), p. 022316.

[15] Craig Gidney and Martin Ekerå. “How to factor 2048 bit RSA integers in 8 hours using 20 million noisy qubits”. In: *Quantum* 5 (Apr. 2021), p. 433. ISSN: 2521-327X. DOI: 10.22331/q-2021-04-15-433. URL: http://dx.doi.org/10.22331/q-2021-04-15-433.

[16] H. Bombin and M. A. Martin-Delgado. “Optimal resources for topological two-dimensional stabilizer codes: Comparative study”. In: *Physical Review A* 76.1 (July 2007). ISSN: 1094-1622. DOI: 10.1103/physreva.76.012305. URL: http://dx.doi.org/10.1103/PhysRevA.76.012305.

[17] David S. Wang, Austin G. Fowler, and Lloyd C. L. Hollenberg. “Surface code quantum computing with error rates over 1%”. In: *Phys. Rev. A* 83 (2 Feb. 2011), 020302(R). DOI: 10.1103/PhysRevA.83.020302. URL: https://link.aps.org/doi/10.1103/PhysRevA.83.020302.

[18] Andrew J. Landahl, Jonas T. Anderson, and Patrick R. Rice. *Fault-tolerant quantum computing with color codes*. 2011. arXiv: 1108.5738 [quant-ph]. URL: https://arxiv.org/abs/1108.5738.

[19] Austin G. Fowler. “Two-dimensional color-code quantum computation”. In: *Physical Review A* 83.4 (Apr. 2011). ISSN: 1094-1622. DOI: 10.1103/physreva.83.042310. URL: http://dx.doi.org/10.1103/PhysRevA.83.042310.

[20] N. Lacroix et al. “Scaling and logic in the colour code on a superconducting quantum processor”. In: *Nature* 645.8081 (May 2025), pp. 614–619. ISSN: 1476-4687. DOI: 10.1038/s41586-025-09061-4. URL: http://dx.doi.org/10.1038/s41586-025-09061-4.

[21] Sergey Bravyi, David Poulin, and Barbara Terhal. “Tradeoffs for Reliable Quantum Information Storage in 2D Systems”. In: *Physical Review Letters* 104.5 (Feb. 2010). ISSN: 1079-7114. DOI: 10.1103/physrevlett.104.050503. URL: http://dx.doi.org/10.1103/PhysRevLett.104.050503.

[22] Sergey Bravyi et al. “High-threshold and low-overhead fault-tolerant quantum memory”. In: *Nature* 627.8005 (2024), pp. 778–782.

[23] Eric Dennis et al. “Topological quantum memory”. In: *Journal of Mathematical Physics* 43.9 (2002), pp. 4452–4505.

[24] Jack Edmonds. “Paths, trees, and flowers”. In: *Canadian Journal of mathematics* 17 (1965), pp. 449–467.

[25] Oscar Higgott. “PyMatching: A Python package for decoding quantum codes with minimum-weight perfect matching”. In: *ACM Transactions on Quantum Computing* 3.3 (2022), pp. 1–16.

[26] Nicolas Delfosse and Naomi H. Nickerson. “Almost-linear time decoding algorithm for topological codes”. In: *Quantum* 5 (Dec. 2021), p. 595. ISSN: 2521-327X. DOI: 10.22331/q-2021-12-02-595. URL: http://dx.doi.org/10.22331/q-2021-12-02-595.

[27] Daniel Gottesman. “The Heisenberg representation of quantum computers”. In: *Group22: Proceedings of the XXII International Colloquium on Group Theoretical Methods in Physics*. arXiv preprint quant-ph/9807006. International Press. 1999, pp. 32–43.

[28] Scott Aaronson and Daniel Gottesman. “Improved simulation of stabilizer circuits”. In: *Physical Review A* 70.5 (2004), p. 052328. DOI: 10.1103/PhysRevA.70.052328.

[29] Cirq Developers. *Cirq*. https://zenodo.org/doi/10.5281/zenodo.4062499. DOI: 10.5281/ZENODO.4062499. Python package for writing, manipulating, and running quantum circuits on quantum computers and simulators. Aug. 2025.